

SC2002: Object-Oriented Design & Programming

Chapters 1–8 Lecture Notes

Contents

1	Object-Oriented Modelling	2
1.1	Introduction	2
1.1.1	Learning Objectives	2
1.2	Programming Paradigms	2
1.2.1	Unstructured Programming	2
1.2.2	Procedural Programming	3
1.2.3	Object-Oriented Programming	3
1.3	Evolution of Programming Languages	3
1.3.1	Language Popularity Trends	4
1.4	Major Programming Languages	4
1.4.1	The C Language	4
1.4.2	The C++ Language	5
1.4.3	The Java Language	5
1.4.4	The C# Language	6
1.4.5	Procedural vs. Object-Oriented: A Comparison	6
1.5	Classes and Objects	8
1.5.1	What is an Object?	8
1.5.2	Four Basic Components of Object-Oriented Model	9
1.5.3	Four Main Concepts of Object-Oriented Programming	9
2	Class and Object	14
2.0.1	Learning Objectives	14
2.1	Classes and Objects in Java	14
2.1.1	What is an Object?	14
2.1.2	What is a Class?	14
2.2	Class Definition in Java	15
2.2.1	Syntax	15
2.2.2	The Rectangle Example	15
2.2.3	Creating Objects	16
2.3	Constructors	16
2.4	Message Sending	17
2.5	Copying Objects	18
2.6	The Keyword <code>this</code>	18
2.7	Accessors and Mutators	19
2.8	The Keyword <code>static</code>	19
2.8.1	Static Variables (Class Variables)	19
2.8.2	Static Methods (Class Methods)	20
2.8.3	Class Constants	20

2.8.4	Static vs. Instance: Summary	20
2.9	Encapsulation and Information Hiding	21
2.10	Object Composition	21
2.11	The <code>String</code> Class	22
2.12	Java Naming Conventions	23
3	Inheritance	24
3.0.1	Learning Objectives	24
3.1	Inheritance	24
3.1.1	Inheritance Hierarchy Example: Shapes	25
3.1.2	Subclass Definition Rules	25
3.2	Method Overloading	26
3.3	Method Overriding	26
3.3.1	Multi-level Inheritance and Overriding: Disc	26
3.3.2	Resolving Method Calls	27
3.4	Visibility Modifiers	27
3.4.1	Java Packages	27
3.5	Final Classes and Methods	28
3.6	Abstract Classes and Methods	28
3.7	Multiple Inheritance and Interfaces	29
3.7.1	Why Multiple Inheritance Is Needed	29
3.7.2	Interfaces	29
3.7.3	Derived Interfaces	29
3.7.4	Abstract Classes vs. Interfaces	30
3.7.5	Multiple Inheritance via Interfaces in Code	30
4	Exception Handling	31
4.0.1	Learning Objectives	31
4.1	Error Handling	31
4.1.1	Naive Error Handling vs. Exception Handling	31
4.2	Java's Exception Handling Mechanism	32
4.2.1	The <code>try/throw/catch/finally</code> Syntax	32
4.3	Java's Exception Hierarchy	32
4.3.1	Commonly Used Predefined Exceptions	33
4.4	Using Exception Classes	33
4.4.1	The <code>Exception</code> Class	33
4.4.2	Checked vs. Unchecked Exceptions	34
4.4.3	Throwing Exceptions from Methods	34
4.4.4	Exception Propagation	34
4.4.5	Multiple <code>catch</code> Blocks	35
4.5	Creating Custom Exception Classes	35
5	Polymorphism	37
5.0.1	Learning Objectives	37
5.1	Polymorphism	37
5.1.1	Method Overriding: Replace vs. Refine	37
5.1.2	Classic Polymorphism Pattern	38
5.1.3	The Liskov Substitution Principle	38
5.2	Binding	39
5.2.1	Static Binding (Early Binding)	39
5.2.2	Dynamic Binding (Late Binding)	39
5.3	Object Typecasting	40

5.3.1	Upcasting	40
5.3.2	Downcasting	40
5.3.3	The <code>instanceof</code> Operator	41
5.3.4	Reference Type vs. Object Type: Compile-Time and Runtime Checks . .	41
5.4	Benefits of Polymorphism	42
5.4.1	Simplicity	42
5.4.2	Extensibility	42
5.5	Three Ways of Method Overriding	42
5.6	Extended Examples	43
5.6.1	Interface-Based Polymorphism: <code>IAntimal</code>	43
5.6.2	Abstract Class Hierarchy: <code>LivingThings</code>	43
6	UML Class Diagrams and Class Relationships	45
6.1	Why Model?	45
6.2	The Unified Modeling Language (UML)	45
6.2.1	UML Diagram Types	45
6.3	Class Diagrams	46
6.3.1	Class Notation	46
6.4	Class Relationships	47
6.4.1	Dependency	47
6.4.2	Association	47
6.4.3	Aggregation and Composition	49
6.4.4	Generalisation (Inheritance)	50
6.4.5	Realisation (Interface Implementation)	50
6.5	Abstract Classes and Interfaces in Class Diagrams	51
6.6	Object Diagrams	51
6.7	More about Class Diagram Notation	52
6.7.1	Visibility and Static Members	52
6.7.2	Enumerations	52
6.7.3	Constraints	52
6.8	Class Stereotypes	52
7	UML Sequence Diagrams	54
7.1	Introduction	54
7.1.1	Learning Objectives	54
7.2	Sequence Diagram Elements	54
7.2.1	The Frame	54
7.2.2	Lifelines	54
7.2.3	Messages	54
7.2.4	Activation Boxes (Execution Specifications)	55
7.2.5	Self-Messages and Recursive Messages	55
7.2.6	Object Creation and Destruction	55
7.2.7	Message Specification and Parameter Modes	55
7.2.8	Found and Lost Messages	56
7.3	Interaction Fragments (Combined Fragments)	56
7.3.1	<code>opt</code> — Optional Fragment	56
7.3.2	<code>alt</code> — Alternative Fragment	57
7.3.3	<code>loop</code> — Loop Fragment	57
7.3.4	<code>ref</code> — Interaction Use	58
7.4	Diagram to Code	58
7.4.1	Worked Example: <code>CalculatePrice</code>	58

8	Modelling OO Applications	61
8.1	Introduction	61
8.1.1	Learning Objectives	61
8.2	Modelling Real-World Systems	61
8.2.1	Identifying Objects from Requirements	61
8.2.2	Class Stereotypes	61
8.3	Short Example: Professor–Student–Course	62
8.3.1	Requirements Passage	62
8.3.2	Step 1 – Class Diagram	62
8.3.3	Step 2 – Sequence Diagram: <code>sd check mentee result</code>	63
8.4	Case Study: Blackjack	63
8.4.1	Requirements Summary	63
8.4.2	Step 1 — Identify Candidate Classes (Noun Extraction)	64
8.4.3	Step 2 — Process of Elimination	65
8.4.4	Step 3 — Identify Class Responsibilities (Verb Extraction)	65
8.4.5	Step 4 — CRC Cards	66
8.4.6	Step 5 — UML Class Diagram	67
8.4.7	What’s Next: From Design to Code	67
9	Design Principles	70
9.1	Introduction	70
9.1.1	Learning Objectives	70
9.2	Characteristics of Good and Bad Design	70
9.2.1	Symptoms of Rotting Design	70
9.3	Object-Oriented Design Goals	71
9.3.1	Class Design Guidelines	71
9.3.2	Designing for Change: Encapsulation, Coupling, and Cohesion	71
9.3.3	Example — Data Persistence	72
9.3.4	Example — Finance Index Display	72
9.3.5	Example — Bank Account (Interfaces for Loose Coupling)	73
9.3.6	Inheritance vs. Delegation (Composition)	73
9.3.7	Inheritance vs. Role	73
9.4	SOLID Design Principles	74
9.4.1	Single Responsibility Principle (SRP)	74
9.4.2	Open–Closed Principle (OCP)	74
9.4.3	Liskov Substitution Principle (LSP)	75
9.4.4	Interface Segregation Principle (ISP)	75
9.4.5	Dependency Inversion Principle (DIP)	76
9.5	Design Patterns (Overview)	76
9.6	Summary	77
10	Modern Java Enhancements	78
10.1	Introduction to Generics (Java 5)	78
10.1.1	Motivation: The Problem Without Generics	78
10.1.2	Generics: Type-Safe Collections	78
10.1.3	Custom Generic Classes	79
10.1.4	Wildcards	80
10.1.5	Type Parameter Naming Conventions	80
10.2	Collections Enhancements (Java 5+)	80
10.3	Enhanced For Loop (Java 5)	81
10.3.1	Basic For Loop (Index-Based)	81
10.3.2	Enhanced For-Each Loop	81

10.3.3	Iterator (Safe Element Removal)	82
10.3.4	Stream API <code>forEach</code> (Functional, Java 8)	82
10.4	Lambda Expressions (Java 8)	82
10.4.1	Motivation	82
10.4.2	Syntax	82
10.4.3	Functional Interfaces	83
10.4.4	Method References	83
10.5	Streams API (Java 8)	84
10.5.1	What is a Stream?	84
10.5.2	Stream Operations	84
10.6	Default Methods in Interfaces (Java 8)	85
10.6.1	What is a Default Method?	85
10.6.2	Why Are Default Methods Needed?	85
10.6.3	Resolving Default Method Conflicts (Multiple Inheritance)	86
10.6.4	Static Methods in Interfaces	86
10.7	Enhanced Switch (Java 12+)	87
10.7.1	Limitations of the Traditional Switch	87
10.7.2	Enhanced Switch Expression	87
10.7.3	The <code>yield</code> Statement	87
10.7.4	Using Enhanced Switch with Enums	87
10.8	Summary	88

Chapter 1

Object-Oriented Modelling

1.1 Introduction

This chapter introduces the fundamental concepts of object-oriented programming (OOP) and contrasts them with procedural programming paradigms. Understanding these foundational concepts is critical for developing robust, maintainable software systems using modern programming languages.

1.1.1 Learning Objectives

After completing this chapter, students should be able to:

- Differentiate between procedural and object-oriented programming paradigms
- Explain core object-oriented concepts including abstraction, encapsulation, inheritance, and polymorphism
- Understand the evolution and comparative strengths of major programming languages

1.2 Programming Paradigms

Programming languages can be classified into several paradigms based on their fundamental approach to organizing and executing code. The three primary paradigms discussed in this course are:

1.2.1 Unstructured Programming

Definition 1.2.1 (Unstructured Programming). Unstructured programming (UP) is an early programming paradigm characterized by the use of `GOTO` statements and lack of structured control flow. Programs written in this style can be difficult to read, maintain, and debug due to arbitrary jumps in execution.

Examples of unstructured programming languages include Assembly language, early versions of COBOL, and BASIC. Consider this BASIC example demonstrating unstructured control flow:

1.1: Unstructured BASIC code example

```
1 10 INPUT guess
2 20 GOTO 40
3 30 PRINT "This line is skipped"
4 40 IF guess = answer THEN GOTO 60
5 50 GOTO 10
6 60 PRINT "You've won a million"
```

1.2.2 Procedural Programming

Definition 1.2.2 (Procedural Programming). Procedural programming (PP), also known as imperative programming, is based upon the concept of procedure calls or functions. Programs are structured as a sequence of statements that represent commands, organized into reusable procedures that operate on data.

Key Characteristics of Procedural Programming

Procedural programming languages feature:

- Sequence, selection, and iteration control structures
- Use of assignment statements to change values of variables in memory
- Functions/procedures that can be called with parameters
- Global and local variable scopes

Examples include C, Pascal, Ada, and Fortran. Procedural programming was developed to address the maintainability issues of unstructured code by introducing:

- **Sequence:** Statements execute in order
- **Selection:** Conditional execution (**if-else** statements)
- **Iteration:** Loops (**for**, **while**)

1.2.3 Object-Oriented Programming

Definition 1.2.3 (Object-Oriented Programming). Object-oriented programming (OOP) is based on the notion of *objects*, which can be described as collections of memory locations together with all operations that can change the values of those memory locations. Computation is represented as interaction among, or communication between, objects.

Fundamental Entity in OOP

The fundamental entity in OOP is the **object**, which:

- Receives and sends messages
- Performs computations
- Has a local state that it can modify

Problems are solved by objects sending messages to one another.

Examples of object-oriented languages include Simula, Smalltalk, C++, Java, C#, and Objective-C.

1.3 Evolution of Programming Languages

The history of programming languages reflects the evolution from low-level, machine-oriented code to high-level, problem-oriented abstractions. Key milestones include:

- **1950s–1960s:** Fortran, COBOL, LISP, Algol 60 first high-level languages
- **1960s–1970s:** Simula (1960s), Smalltalk (1971) first object-oriented languages

- **1970s:** C (1972), Pascal procedural programming reaches maturity
- **1980s:** C++ (1984), Ada object-oriented extensions to procedural languages
- **1990s:** Java (1995), C# (2000) modern OOP languages with platform independence

1.3.1 Language Popularity Trends

According to the TIOBE Programming Community Index (2011–2015), object-oriented languages have dominated the landscape:

- Java consistently ranked #1 (19.4% in August 2011)
- C maintained strong presence (#2, 17.4%)
- C++ remained in top 3 (8.4%)
- C#, Objective-C, Python showed significant growth

The shift toward object-oriented languages reflects industry demands for:

- Code reusability
- Easier maintenance and modification
- Better modeling of real-world problems
- Support for large-scale software development

1.4 Major Programming Languages

1.4.1 The C Language

Developed by Brian Kernighan and Dennis Ritchie at Bell Labs for implementing the UNIX operating system, C remains one of the most influential programming languages.

Key Characteristics:

- Provides access to low-level data and memory
- Generates efficient compiled code
- Supports system programming
- Enhanced by the ANSI standard for portability

C Code Readability

Programs in C are often expressed with compact code at the expense of readability. For example, the declaration:

```
char (* (* x()) []) ()
```

declares `x` as a function returning a pointer to an array of pointers to functions returning `char`.

Strengths:

- Excellent for construction of portable systems programs
- Close to hardware, efficient execution
- Widely supported compilers and tools

1.4.2 The C++ Language

Definition 1.4.1 (C++). C++ was developed by Bjarne Stroustrup at AT&T as a superset of C that includes object-oriented mechanisms without compromising the efficiency and simplicity of C.

Key Features:

- Strict type-checking for improved safety
- Class mechanism for data abstraction and encapsulation
- Support for both procedural and object-oriented paradigms
- Templates for generic programming

C++ is Not Pure OOP

C++ does not force you to use its OOP features. You can write code that uses only C++'s non-OOP features, making it a hybrid language. This flexibility is both a strength (gradual migration from C) and a weakness (inconsistent coding styles).

Remark 1.4.2. C++ has been described as “a black Firebird, the all macho car that comes with optional seatbelt (lint)” emphasizing its power and performance, but also the responsibility it places on the programmer to write safe code.

1.4.3 The Java Language

Definition 1.4.3 (Java). Java was developed by Sun Microsystems (led by James Gosling) initially for embedded consumer goods, but became popular for web development. It is based on C/C++ syntax but is smaller, more reliable, and more portable.

Key Restrictions for Safety:

- No pointers (eliminates pointer arithmetic errors)
- No multiple inheritance (reduces complexity)
- Automatic garbage collection (prevents memory leaks)
- Platform-independent bytecode (“Compile Once, Run Anywhere”)

Execution Model: Java uses a two-stage execution process:

1. **Compilation:** Java source code (`.java`) is compiled by the Java Compiler into platform-independent bytecode (`.class`)
2. **Execution:** The Java Virtual Machine (JVM) executes the bytecode, using a Just-In-Time (JIT) compiler to convert it to native machine code at runtime

Java's Platform Independence

The Java Runtime Environment (JRE) provides:

- Java Virtual Machine (JVM) for bytecode execution
- Just-In-Time compiler for performance optimization
- Standard libraries for cross-platform functionality

This enables the “write once, run anywhere” philosophy.

Remark 1.4.4. Java has been humorously described as an “all-terrain very slow vehicle” noting its portability at the expense of execution speed compared to compiled languages like C++. However, modern JIT compilers have significantly improved Java's performance.

1.4.4 The C# Language

Definition 1.4.5 (C#). C# (“C Sharp”) was developed by Microsoft, led by Anders Hejlsberg (creator of Turbo Pascal and Delphi). It is based on C++ but incorporates many features similar to Java.

Key Features:

- No pointers (safe memory management)
- No multiple inheritance (single inheritance with interfaces)
- No global variables (everything must be in a class)
- Strictly object-oriented (cannot write non-OOP code)
- Automatic garbage collection
- Multi-threading support
- Cross-platform via .NET framework

1.4.5 Procedural vs. Object-Oriented: A Comparison

To illustrate the difference between procedural and object-oriented approaches, consider a simple factorial calculation program.

1.4.5.1 C Implementation (Procedural)

1.2: Factorial in C (procedural approach)

```
1 #include <stdio.h>
2
3 int getFact(int n) {
```

```

4   int c, fact = 1;
5   if (n < 0)
6       printf("Number should be non-negative.");
7   else
8       for (c = 1; c <= n; c++)
9           fact = fact * c;
10  return fact;
11 }
12
13 int main()
14 {
15     int n = 1;
16     printf("Enter a number to calculate its factorial\n");
17     scanf("%d", &n);
18     printf("Factorial of %d = %d\n", n, getFact(n));
19     return 0;
20 }
21 // save as <anyname>.c

```

1.4.5.2 Java Implementation (Object-Oriented)

1.3: Factorial in Java (object-oriented approach)

```

1  import java.util.Scanner;
2
3  class Factorial // class name
4  {
5      public static int getFact(int n) {
6          int c, fact = 1;
7          if (n < 0)
8              System.out.println("Number should be non-negative.");
9          else
10             for (c = 1; c <= n; c++)
11                 fact = fact * c;
12             return fact;
13     }
14
15     public static void main(String args[])
16     {
17         int n = 1;
18         System.out.println("Enter an integer to calculate its factorial");
19         Scanner in = new Scanner(System.in);
20         n = in.nextInt();
21         System.out.println("Factorial of " + n + " is = " + getFact(n));
22     }
23 }
24 // must save as <class name>.java => Factorial.java

```

Key Structural Differences

Comparing C and Java:

- **C:** Functions exist independently; can use global variables; single `main()` entry point
- **Java:** Everything must be inside a class; no global variables; `main()` is a static method of a class; filename must match the public class name

1.4.5.3 Architectural Differences

Procedural Programming (C):

- Functions operate on global data
- Control flow moves between functions
- Data and operations are separate
- Functions access shared global state

Object-Oriented Programming (Java):

- Data (attributes) and operations (methods) are bundled into objects
- Objects communicate via messages (method calls)
- Each object manages its own state
- No global data encapsulation protects object state

1.5 Classes and Objects

1.5.1 What is an Object?

Objects are fundamental to object-oriented programming. They model real-world entities and concepts.

Definition 1.5.1 (Object). An object is an entity that contains both the *attributes* that describe the state of a real-world object and the *actions* (methods) that are associated with the real-world object.

Characteristics of Objects:

- **Identity:** A unique identifier that distinguishes it from other objects
- **State:** The set of attribute values that define the current condition of the object
- **Behavior:** The operations or methods that the object can perform

Example 1.5.2 (Person Object). Consider modeling a person, Barack Obama, as an object:

- **Identity:** Barack Obama (unique individual)
- **State/Attributes:**
 - height: 187 cm
 - weight: 81.6 kg
 - gender: M
 - age: 49
 - wealth: [amount]
- **Behavior/Methods:**
 - Eat
 - Study/Sleep

Example 1.5.3 (Car Object). A car can be modeled as an object with:

- **Identity:** Machine serial number
- **State/Properties:** Color, maximum speed, fuel level, etc.
- **Behavior/Methods:** Turn, brake, accelerate

1.5.2 Four Basic Components of Object-Oriented Model

Definition 1.5.4 (Objects). Objects are entities that contain both the attributes that describe the state of a real-world object and the actions that are associated with the real-world object.

Definition 1.5.5 (Messages). Messages are requests from one object to another for the receiving object to produce some desired result. They represent the communication mechanism between objects.

Definition 1.5.6 (Methods). Methods are descriptions of operations or actions that an object performs when it receives a message.

Definition 1.5.7 (Classes). A class is a template for objects which consists of methods and state descriptions that objects belong to. It defines the blueprint from which individual objects are created.

Class vs. Object Relationship

- A **class** defines the structure and behavior (template/blueprint)
- An **object** is a specific instance of a class with actual values
- Example: **Person** is a class; **Obama** is an object (instance) of the **Person** class

1.5.3 Four Main Concepts of Object-Oriented Programming

The four pillars of object-oriented programming are:

1. Abstraction
2. Encapsulation / Information Hiding
3. Inheritance
4. Polymorphism

1.5.3.1 Abstraction

Definition 1.5.8 (Abstraction). “An abstraction denotes the essential characteristics of an object that distinguish it from all other kinds of objects and thus provide crisply defined conceptual boundaries, relative to the perspective of the viewer.”

— Grady Booch, *Object-Oriented Analysis and Design with Applications*, 1994

Abstraction focuses on what an object does rather than how it does it. It involves:

- Identifying essential characteristics
- Hiding implementation details
- Providing a simplified interface

State of an Object: The state “encompasses all of the (usually static) properties of the object plus the current (usually dynamic) values of each of these properties” (Booch, 1994).

Definition 1.5.9 (Attributes). Attributes are the data or variables that characterize the state of an object.

Behavior of an Object: Methods describe the behavior associated with an object. They represent:

- Services that the object provides to other objects
- Operations that the object may perform upon other objects
- Interfaces provided by the object

Example 1.5.10 (Car Abstraction). Consider an Aston Martin car:

- **Attributes:** Color (Black), Number of Doors (2), Engine Size, etc.
- **Behavior:** Steer, ChangeGear, Accelerate, Brake
- **Abstraction Barrier:** The client (driver) interacts through the steering wheel, pedals, and gear shift they don’t need to understand the internal combustion process, transmission mechanics, or brake hydraulics

1.5.3.2 Encapsulation and Information Hiding

Definition 1.5.11 (Encapsulation). Encapsulation builds a barrier to protect an object’s private data. Access to private data can be done only through the public methods of the object’s class (e.g., get and set methods).

Definition 1.5.12 (Information Hiding). Information hiding conceals the details or implementation of the class from users. Users only need to know *what* a class does and *how* to call the methods to perform tasks, not the implementation details.

Benefits of Encapsulation

Encapsulation provides:

- **Data protection:** Prevents unauthorized access and modification
- **Interface stability:** Internal implementation can change without affecting users
- **Modularity:** Components can be developed and maintained independently

Encapsulation in Practice

Think of a vending machine:

- The user sends a message: `getACoke()`
- The machine’s internal state (number of cokes, money stored) is hidden
- The user receives the result (a Coke) without knowing the internal mechanism
- The user cannot call `getRecipe()` that method is not exposed

Real-World Analogy: Consider how different systems hide complexity:

- **Car:** Driver doesn't see the engine, transmission, or brake system internals
- **Human body:** The brain's decision-making processes are hidden from conscious awareness
- **Computer:** The CPU's microarchitecture is hidden behind the instruction set

1.5.3.3 Inheritance

Definition 1.5.13 (Inheritance). Inheritance is a mechanism that defines a new class which inherits the properties and behaviors (methods) of a parent class. It supports code reuse and establishes relationships between classes.

Terminology:

- **Superclass** or **base class** (parent): The class from which another class inherits structures and/or behaviors
- **Subclass** or **derived class** (child): A class that inherits from one or more other classes
- **“is-a” relationship:** A subclass object “is a” type of superclass object

Example 1.5.14 (Employee Hierarchy). Consider an **Employee** class hierarchy:

Superclass: Employee

- Attributes: `firstName`, `lastName`, `socialSecurityNo`
- Methods: `earnings()`

Subclass: SalariedEmployee (inherits from **Employee**)

- Additional Attributes: `weeklySalary`
- Additional Methods: `overtimePay()`
- Inherits: `firstName`, `lastName`, `socialSecurityNo`, `earnings()`

Subclass: HourlyEmployee (inherits from **Employee**)

- Additional Attributes: `wagePerHour`, `hours`
- Overridden Methods: `earnings()` reimplemented to calculate hourly wages
- Inherits: `firstName`, `lastName`, `socialSecurityNo`

Benefits of Inheritance

- **Code reuse:** Common functionality defined once in the parent class
- **Extensibility:** New classes can be created by extending existing ones
- **Polymorphism:** Subclasses can be treated as instances of their superclass
- **Maintainability:** Changes to common behavior require updates in only one place

Method Overriding: Any inherited behavior may be redefined in a subclass, superseding the inherited definition. In Example 1.5.14, **HourlyEmployee** overrides the `earnings()` method to implement a different calculation than the one in **Employee**.

Multiple Inheritance:

Definition 1.5.15 (Multiple Inheritance). Multiple inheritance allows a class to inherit from more than one superclass.

Example 1.5.16 (Student-Employee with Multiple Inheritance). A `StudentEmployee` might inherit from both `Student` and `Employee`:

Student class:

- Attributes: `name`, `address`, `age`
- Methods: `printExamRes()`

Employee class:

- Attributes: `name`, `address`, `salary`, `age`
- Methods: `earnings()`

`StudentEmployee` inherits from both, but must resolve conflicts (e.g., which `name` and `address` to use).

Problems with Multiple Inheritance

When a class inherits from multiple superclasses that have properties or methods with the same name, ambiguity arises. Solutions include:

- User-specified precedence (as in Smalltalk)
- Renaming one property/method
- Accepting both if signatures are different (e.g., `print(String s)` vs. `print(Char c)`)

Many modern languages (Java, C#) avoid this complexity by **not supporting** multiple inheritance of classes, using interfaces instead.

1.5.3.4 Polymorphism

Definition 1.5.17 (Polymorphism). Polymorphism is a Greek word meaning “many shapes” or “multiple forms.” In object-oriented programming, it means that the same message can be sent to different objects, and each object performs the operation appropriate to its class doing similar things differently.

Power of Polymorphism

Polymorphism is considered the most powerful concept of object-oriented programming because:

- The sending object does not need to know the class of the receiving object
- The sending object does not need to know how the receiving object will respond
- Code can be written generically to work with any object in a class hierarchy
- New classes can be added without modifying existing code

Example 1.5.18 (Animal Movement Polymorphism). Consider a `call` function that tells an animal to move:

```
1 void call(Animal animal) {  
2     animal.move();  
3 }
```

The `Animal` superclass defines a `move()` method, which is inherited and reimplemented by each subclass:

- `Eagle.move()` flies through the air
- `Tiger.move()` runs on the ground
- `Shark.move()` swims through water

The `call` function works with *any* `Animal` subclass without modification. When `animal.move()` is invoked:

- If `animal` is an `Eagle`, it flies
- If `animal` is a `Tiger`, it runs
- If `animal` is a `Shark`, it swims

This is polymorphism the same method call produces different behaviors based on the actual object type.

Chapter 2

Class and Object

Building on the object-oriented modelling concepts introduced in Chapter 1.5, this chapter provides a detailed treatment of how classes and objects are defined and used in Java. Topics include class definition syntax, constructors, message sending, object copying, the `this` keyword, accessors and mutators, the `static` keyword, encapsulation, object composition, and the `String` class.

2.0.1 Learning Objectives

After completing this chapter, you should be able to:

- Describe classes and objects in Object-Oriented Programming (OOP).
- Define a class in OOP and translate a UML class diagram to Java code.
- Explain the concept of message sending and how it maps to method invocation.
- Explain how to correctly copy objects (shallow reference copy vs. deep copy).
- Explain the `this` keyword and its three uses.
- Describe accessors (getters) and mutators (setters).
- Describe the `static` keyword and distinguish class variables/methods from instance variables/methods.
- Differentiate between encapsulation and information hiding.
- Describe object composition and the “has-a” relationship.
- Explain the `String` class and its immutability.

2.1 Classes and Objects in Java

2.1.1 What is an Object?

Recall from Chapter 1.5 that every object in OOP has three fundamental characteristics:

Definition 2.1.1 (Object Characteristics). An **object** has:

1. **Identity** — a unique identifier (Object Identifier, OID) that distinguishes it from every other object, even one with identical state.
2. **State** — a set of *data properties* (attributes) whose values describe the object at a point in time (e.g. gender, age, monthly salary for a `Person`).
3. **Behaviour** — the set of operations the object can perform, implemented as *methods* (e.g. `eat()`, `walk()`, `sleep()`, `talk()`).

2.1.2 What is a Class?

Definition 2.1.2 (Class). A **class** defines the blueprint or structure for creating objects. In Java, a class contains:

- **Data properties** (instance variables / attributes) — the state template.

- **Methods** — the behaviour template.

An **object** is a specific *instance* of a class, created by *instantiation*.

Classes are modelled using UML *Class Diagrams*, which have three compartments: class name, data attributes, and methods. The access modifiers + (public) and - (private) appear in front of each member.

Example 2.1.3 (Person Class Diagram). The **Person** class has four private attributes and four public methods:

Person
- name: String
- gender: char
- age: int
- height: double
+ eat(): void
+ walk(): void
+ sleep(): void
+ talk(): void

Multiple **Person** objects (johnObject, billObject, obamaObject) can be instantiated from this single class, each holding its own values for name, age, etc.

2.2 Class Definition in Java

2.2.1 Syntax

The general syntax for defining a class in Java is:

2.1: General class definition syntax

```

1 public class Class_Name {
2     // Instance variable declarations
3     private Type instanceVariable1;
4     // ...
5
6     // Method definitions
7     public ReturnType methodName(ParameterList) {
8         // method body
9     }
10    // ...
11 }
```

- Instance variables are declared **private** to enforce *information hiding*.
- Methods are declared **public** to expose the object's interface.
- The method header format is: **public <static> ReturnType MethodName(ParameterList)**, where **ReturnType** can be a primitive type, a class type, or void.

2.2.2 The Rectangle Example

The **Rectangle** class illustrates the principles of *data abstraction*: we identify the common attributes (width, height) and the common operations (find area, find perimeter) shared by all rectangles.

2.2: Rectangle class definition

```
1 public class Rectangle {
2     // Instance variables
3     private double width;
4     private double height;
5
6     // Instance methods
7     public double findArea() {
8         return width * height;
9     }
10
11     public double findPerimeter() {
12         return (width + height) * 2;
13     }
14 }
```

Key Result: Class and OOP Principles

Declaring instance variables as **private** and methods as **public** is how Java enforces **encapsulation** and **information hiding** — two of the four main OOP concepts introduced in Chapter 1.5.

2.2.3 Creating Objects

Object creation in Java is a two-step process:

1. **Declare** an object reference variable: `ClassName objectRef;`
2. **Allocate** memory and instantiate: `objectRef = new ClassName();`

These two steps are typically combined:

```
1 Rectangle rect = new Rectangle();
```

Definition 2.2.1 (Object Reference vs. Object). An **object reference** (e.g. `rect`) does *not* store the object itself; it stores the *memory address* of the object. Assigning one reference to another (`rect3 = rect2`) does not copy the object — both references then point to the *same* object. An unassigned reference holds the value `null`.

Warning: Reference Assignment is Not Object Copying

The statement `rect2 = rect1;` creates only *one* object with two references pointing to it. Any change made through `rect1` is immediately visible via `rect2`. To obtain an independent copy, a dedicated copy method is required (see Section 2.5).

2.3 Constructors

Definition 2.3.1 (Constructor). A **constructor** is a special method used to *initialise* an object's data properties at creation time. It has the same name as the class and no return type. If no constructor is written, Java provides a default no-argument constructor that does nothing.

Constructors can be *overloaded* (multiple constructors with different parameter lists):

2.3: Rectangle with two constructors

```
1 public class Rectangle {
2     private double width;
3     private double height;
4
5     // No-argument constructor (default values)
6     public Rectangle() {
7         width = 1.0;
8         height = 1.0;
9     }
10
11    // Parameterised constructor
12    public Rectangle(double w, double h) {
13        width = w;
14        height = h;
15    }
16 }
```

Usage:

```
1 Rectangle rect1 = new Rectangle(); // width=1.0, height=1.0
2 Rectangle rect2 = new Rectangle(10.0, 20.0); // width=10.0, height=20.0
```

Remark 2.3.2 (Finalizer). The counterpart to a constructor is the **finalizer** (`finalize()` method in Java), which releases resources when an object is no longer needed. Java's *Garbage Collector* calls the finalizer automatically.

2.4 Message Sending

Definition 2.4.1 (Message Sending). In OOP, **message sending** is the act of asking an object to perform one of its methods. The syntax is:

`objectReference.methodName(argumentList);`

The `objectReference` identifies the *receiver object*, the dot operator separates the receiver from the message, and `argumentList` provides input data (if any).

Warning: NullPointerException

Invoking a method on an object reference that has not been initialised (i.e. is `null`) causes a `NullPointerException` at runtime. Always ensure the reference points to a valid object before calling methods on it.

2.4: RectangleApp — calling instance methods

Example 2.4.2 (Message Sending with Rectangle).

```
1 public class RectangleApp {
2     public static void main(String[] args) {
3         Rectangle rect = new Rectangle(); // width=5.0, height=10.0 (hardcoded in class)
4         System.out.println("Area    = " + rect.findArea());
5         System.out.println("Perimeter = " + rect.findPerimeter());
6     }
7 }
8 // Output:
9 // Area    = 50.0
10 // Perimeter = 30.0
```

2.5 Copying Objects

Because an assignment such as `rect2 = rect1` only copies the reference (address), a *deep copy* requires a dedicated method that allocates a new object and copies all field values:

2.5: `copyObjects()` method in `Rectangle`

```
1 public Rectangle copyObjects() {  
2     return new Rectangle(width, height);  
3 }
```

2.6: Using `copyObjects()`

```
1 Rectangle rect1 = new Rectangle(10.0, 20.0);  
2 Rectangle rect2 = rect1.copyObjects(); // Two independent objects, same values
```

After `copyObjects()`, `rect1` and `rect2` each point to a *separate* memory location containing identical data. Modifying one does not affect the other.

2.6 The Keyword `this`

Definition 2.6.1 (`this` Keyword). The keyword `this` is an implicit reference to the *receiver object* — the object on which the current method is being invoked. It is automatically available inside all instance methods and constructors.

`this` has three primary uses in Java:

1. **Disambiguate instance variables from parameters** with the same name:

```
1 public Rectangle(double width, double height) {  
2     this.width = width; // this.width = instance variable  
3     this.height = height; // height = parameter  
4 }
```

2. **Call another method on the current object:**

```
1 public void print() {  
2     System.out.println("Area    = " + this.findArea());  
3     System.out.println("Perimeter = " + this.findPerimeter());  
4 }
```

(The `this.` prefix is optional when there is no naming ambiguity, but makes the intent explicit.)

3. **Delegate to another constructor** using `this(...)`:

```
1 public Rectangle() {  
2     this(1.0, 1.0); // calls Rectangle(double, double)  
3 }
```

Exam Tip: Constructor Delegation

Using `this(args)` to call another constructor in the same class keeps initialisation logic in one place and avoids code duplication. It must be the *first statement* in the constructor body.

2.7 Accessors and Mutators

Because instance variables are **private**, external code cannot read or write them directly. Instead, well-designed classes provide controlled access through:

Definition 2.7.1 (Accessors and Mutators). • **Accessor** (getter method): returns the value of a private data property. Naming convention: `getXxx()`.

- **Mutator** (setter method): changes the value of a private data property, optionally enforcing data consistency rules. Naming convention: `setXxx(value)`.

2.7: Rectangle with accessors and mutators

```
1 public class Rectangle {
2     private double width;
3     private double height;
4
5     public Rectangle() { this(1.0, 1.0); }
6
7     public Rectangle(double width, double height) {
8         this.width = width;
9         this.height = height;
10    }
11
12    // Mutators
13    public void setWidth(double w) { width = w; }
14    public void setHeight(double h) { height = h; }
15
16    // Accessors
17    public double getWidth() { return width; }
18    public double getHeight() { return height; }
19
20    // Utility methods
21    public double findArea() { return width * height; }
22    public double findPerimeter() { return (width + height) * 2; }
23 }
```

Mutators are particularly important for maintaining *data consistency*. For example, if a business rule states that width must not exceed 100, the mutator can silently cap the value instead of storing an invalid state.

2.8 The Keyword `static`

2.8.1 Static Variables (Class Variables)

Definition 2.8.1 (Static Variable). A **static variable** (class variable) is declared with the **static** keyword. Only a *single copy* of the variable exists for the entire class, shared by all objects. It is allocated in a separate memory location from instance variables.

A classic use case is counting how many objects of a class have been created:

```
1 private static int numRectangles = 0;
2
3 public Rectangle(double w, double h) {
4     width = w;
5     height = h;
6     numRectangles++; // increments the single shared copy
7 }
```

2.8.2 Static Methods (Class Methods)

Definition 2.8.2 (Static Method). A **static method** (class method) belongs to the class rather than to any particular object. It is invoked using the class name:

`ClassName.methodName(args);`

Static methods can only access *static* variables and other static methods directly; they cannot reference instance variables or instance methods (which belong to specific objects).

2.8: Static getter and setter for numRectangles

```
1 public static void setNumRectangles(int number) {
2     numRectangles = number;
3 }
4 public static int getNumRectangles() {
5     return numRectangles;
6 }
```

2.9: Calling a static method

```
1 Rectangle rect1 = new Rectangle(5.0, 10.0);
2 Rectangle rect2 = new Rectangle(50.0, 100.0);
3 System.out.println("Count = " + Rectangle.getNumRectangles()); // prints 2
```

2.8.3 Class Constants

Constants are declared with `static final`. By convention they use UPPER_SNAKE_CASE:

```
1 public static final double MAX_WIDTH = 100;
2 public static final double MAX_HEIGHT = 100;
```

Using named constants instead of *magic numbers* improves readability and makes changes easier (update one declaration rather than every occurrence). Mutators can then enforce these limits:

```
1 public void setWidth(double w) {
2     width = (w >= MAX_WIDTH) ? MAX_WIDTH : w;
3 }
```

2.8.4 Static vs. Instance: Summary

Feature	Instance	Static (Class)
Memory allocation	Per object	Once per class
Accessed via	Object reference	Class name
Can access instance members?	Yes	No
Typical use	Object state/behaviour	Counters, utilities, constants

Warning: Static Methods Cannot Call Instance Methods Directly

A static method has no `this` reference. Calling `findArea()` inside a static method is a compile-time error because the compiler cannot determine *which* object's `width` and `height` to use.

2.9 Encapsulation and Information Hiding

Recall the definitions introduced in Chapter 1.5. This chapter reinforces them with a concrete example.

Definition 2.9.1 (Encapsulation (revisited)). **Encapsulation** bundles data (private instance variables) and the methods that operate on them into a single class, building a barrier that protects the internal state from direct external modification. Access to private data is permitted only through the class’s *public interface* (get/set methods).

Definition 2.9.2 (Information Hiding (revisited)). **Information hiding** means that users of a class need only know *what* it does (the public method signatures — the API) and not *how* it does it (the private implementation details). The private bodies of methods and private helper methods are hidden from users.

Key Result: The Public Interface (API)

The **Application Programming Interface (API)** of a class consists of:

- Public method *headers* (name, parameters, return type).
- Public class *constants*.

The implementation (method bodies, private variables, private methods) is hidden. This separation allows the implementation to change without breaking code that uses the class.

2.10 Object Composition

Definition 2.10.1 (Object Composition). **Object composition** is when one class includes an object of another class as one of its instance variables. In OOP this models the “**has-a**” relationship. The containing object holds an *object reference* to the contained object; they reside in separate memory locations.

Example 2.10.2 (Student has-a Course). A Student has a Course:

2.10: Course class

```
1 public class Course {
2     private int courseId;
3     private int year;
4     private int semester;
5     private int score;
6
7     public Course(int courseId, int year, int semester, int score) {
8         this.courseId = courseId;
9         this.year      = year;
10        this.semester = semester;
11        this.score    = score;
12    }
13
14    public int getCourseId() { return courseId; }
15    public int getYear()    { return year;     }
16    public int getSemester() { return semester; }
17    public int getScore()   { return score;    }
18    // setters omitted for brevity
19 }
```

2.11: Student class using composition

```
1 public class Student {
2     private int studentId;
3     private char gender;
4     private double height;
5     private int age;
6     private Course course; // object reference --- composition
7
8     public Student(int studId, char sex, double studHeight,
9                     int studAge, Course courseIn) {
10         studentId = studId;
11         gender = sex;
12         height = studHeight;
13         age = studAge;
14         course = courseIn; // stores reference to the Course object
15     }
16
17     public void printInfo() {
18         System.out.println("Student ID: " + studentId);
19         System.out.println("Course: " + course.getCourseId());
20         System.out.println("Score: " + course.getScore());
21         // ...
22     }
23 }
```

2.12: Application class

```
1 Course course1 = new Course(103, 2004, 2, 80);
2 Student myStudent1 = new Student(11456, 'M', 1.75, 25, course1);
3 myStudent1.printInfo();
```

Composition can be extended to arrays: a `Student` can hold `Course[] course = new Course[3]` for multiple courses, and a `School` class can hold `Student[] myStudentArray` for a full database of student objects.

Exam Tip: Has-a vs. Is-a

Object composition models the **has-a** relationship (a `Student` has a `Course`). This is distinct from inheritance, which models the **is-a** relationship. Both are fundamental OOP design relationships covered in later chapters.

2.11 The String Class

Definition 2.11.1 (`String`). `java.lang.String` is a built-in Java class that represents a sequence of characters. It is a *reference type* (class type), not a primitive. String literals are enclosed in double quotes (e.g. `"Java Programming"`).

`String` objects can be created in two ways — both are equivalent:

```
1 String aString = new String("Java Programming");
2 String aString = "Java Programming"; // preferred shorthand
```

The variable `aString` holds a *reference address* to the string object, not the characters themselves. Characters are indexed from 0, so `aString.charAt(0)` returns `'J'`.

Key Result: Strings are Immutable

A `String` object in Java is **immutable** — its content cannot be changed after creation. An assignment like `aString = "Java";` does *not* modify the original string object; it creates a new `String` object "Java" and makes `aString` point to it. The original "Java Programming" object becomes unreachable and is eventually garbage-collected.

2.12 Java Naming Conventions

Consistent naming is important for code readability. Java uses letter-case conventions:

Convention	Format	Used for
Lowercase	<code>packagename</code>	Package names
UPPER_SNAKE_CASE	<code>MAX_HOURS</code>	Constants, enums
UpperCamelCase	<code>CustomerAccount</code>	Classes, interfaces
lowerCamelCase	<code>customerFirstName</code>	Methods, variables

Chapter 3

Inheritance

3.0.1 Learning Objectives

After completing this chapter, you should be able to:

- Explain inheritance in Object-Oriented Programming (OOP).
- Describe method overloading and method overriding.
- Describe visibility modifiers.
- Write `final` classes and methods.
- Describe abstract classes and methods.
- Describe multiple inheritance and interfaces.

3.1 Inheritance

Definition 3.1.1 (Inheritance). **Inheritance** is an OOP mechanism that allows a new class (*subclass*) to be derived from an existing class (*superclass*) by absorbing its attributes and behaviours, and optionally adding new capabilities.

Inheritance promotes **code reuse** and significantly reduces programming effort when implementing new classes. It captures the “**is-a**” relationship between classes: if class **Student** extends class **Person**, then a **Student is-a Person**.

Key terminology:

- **Superclass** (also: parent class, base class, generalised class) the class being extended.
- **Subclass** (also: derived class, child class, specialised class) the class that extends the superclass.
- In Java, the root ancestor of all classes is `java.lang.Object`.

The generalisation/specialisation relationship means the superclass captures features shared by all subclasses, while each subclass specialises that behaviour with additional attributes and methods.

Key Result: What a Subclass Inherits

A subclass inherits *all* instance variables and methods of the superclass, **except** those declared **private** (access control applies see Section 3.4).

3.1.1 Inheritance Hierarchy Example: Shapes

Consider a `Shape` superclass with subclasses `Rectangle` and `Circle`. `Shape` holds the common attributes `xPos` and `yPos`, while each subclass adds its own geometry.

Superclass: Shape

```
1 public class Shape {
2     private double xPos;
3     private double yPos;
4
5     public Shape() { xPos = 0; yPos = 0; }
6     public Shape(double xCoor, double yCoor) {
7         xPos = xCoor; yPos = yCoor;
8     }
9     public double getXPos() { return xPos; }
10    public double getYPos() { return yPos; }
11    public void print() {
12        System.out.println("[x, y] = [" + xPos + ", " + yPos + "]");
13    }
14 }
```

Subclass: Rectangle uses the `extends` keyword and calls the superclass constructor via `super()`:

```
1 public class Rectangle extends Shape {
2     private double width;
3     private double height;
4
5     public Rectangle() { super(); width = 0; height = 0; }
6     public Rectangle(double xCoor, double yCoor, double w, double h) {
7         super(xCoor, yCoor); width = w; height = h;
8     }
9     public double getWidth() { return width; }
10    public double getHeight() { return height; }
11    public double findArea() { return width * height; }
12    public double findPerimeter() { return 2 * (width + height); }
13    public void print() {
14        System.out.println("Rectangle print() method:");
15        System.out.print("Center "); super.print(); // call superclass method
16        System.out.println("Width = " + width + "; Height = " + height);
17        System.out.println("Perimeter = " + findPerimeter());
18        System.out.println("Area = " + findArea());
19    }
20 }
```

3.1.2 Subclass Definition Rules

- A subclass is declared with `extends SuperclassName`.
- The keyword `super` serves two purposes:
 1. **Calling the superclass constructor:** `super(...)` must be the *first statement* inside the subclass constructor.
 2. **Calling a superclass method:** e.g. `super.print()` invokes the superclass version of an overridden method.

Warning: `super()` Must Be First

If you call `super()` in a subclass constructor, it *must* be the very first statement. Placing any other statement before it will cause a compile-time error.

3.2 Method Overloading

Definition 3.2.1 (Method Overloading). **Method overloading** occurs when multiple methods in the same class share the same name but differ in their *signature* i.e. the number of parameters or the types of parameters.

Overloading is *not* a consequence of inheritance; it can occur within any single class. Its advantage is that related operations can share an intuitive name rather than requiring distinct names like `printString()`, `printInt()`, `printBool()`.

Example 3.2.2 (Overloaded `findMin`).

```
1 public class Minimum {
2     public int findMin(int num1, int num2) { ... }
3     public double findMin(double num1, double num2) { ... }
4     public int findMin(int num1, int num2, int num3) { ... }
5     // public int findMin(int a, int b, int c) { ... } // ERROR: duplicate signature
6 }
```

The last definition would be a *compile-time error* because the signature `(int, int, int)` already exists the parameter *names* are irrelevant to the signature.

Exam Tip: Overloading vs. Overriding

Overloading is resolved at *compile time* based on argument types. Overriding is resolved at *runtime* via dynamic dispatch. Both use the same method name the key difference is whether the signature matches exactly (overriding) or differs (overloading).

3.3 Method Overriding

Definition 3.3.1 (Method Overriding). When a subclass defines a method with *exactly the same signature and return type* as a method in its superclass, the subclass **overrides** that method.

There are two common patterns for overriding:

1. **Refinement** extend the superclass behaviour using `super.methodName()`, then add new logic.
2. **Replacement** completely replace the superclass implementation without calling `super`.

3.3.1 Multi-level Inheritance and Overriding: Disc

Consider `Disc` extends `Circle` extends `Shape`. `Disc` overrides `findArea()` from `Circle` to account for the cylindrical surface area, and overrides `print()` to display thickness and volume:

```
1 public class Disc extends Circle {
2     private double thickness;
3
4     public Disc() { super(); thickness = 0; }
5     public Disc(double xCoor, double yCoor, double rad, double t) {
6         super(xCoor, yCoor, rad); thickness = t;
7     }
8     public double getThickness() { return thickness; }
9
10    // Overrides Circle.findArea() refinement pattern
11    public double findArea() {
12        return 2 * super.findArea() + 2 * Math.PI * getRadius() * thickness;
```

```

13 }
14 public double findVolume() { return super.findArea() * thickness; }
15
16 public void print() { // overriding: refinement
17     System.out.println("Disc print() method:");
18     super.print(); // calls Circle.print()
19     System.out.println("Thickness = " + thickness);
20     System.out.println("Area = " + findArea());
21     System.out.println("Volume = " + findVolume());
22 }
23 }

```

3.3.2 Resolving Method Calls

When a method is called on an object, Java searches for a matching implementation starting at the object's own class, then moves up the hierarchy to each successive superclass until a match is found. If no match is found anywhere in the chain, a compile-time error is reported.

3.4 Visibility Modifiers

Java provides four levels of access control:

- **public** accessible anywhere in the application.
- **private** accessible only within the declaring class.
- **protected** accessible within the declaring class, its subclasses, and any class in the same package.
- **Package (no modifier)** accessible to all classes in the same package, but not to subclasses in other packages.

Modifier	Within class	Same package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
(none)	Y	Y	N	N
private	Y	N	N	N

In UML diagrams, **protected** is denoted with #, **private** with -, and **public** with +.

3.4.1 Java Packages

A **package** groups related classes into a named directory namespace. The package declaration appears as the first statement in a source file:

```

1 package packageOne;
2 public class Class1 { ... }

```

Packages derive from the Internet domain convention, e.g. **sg.edu.ntu.projects**. If no package is declared, classes belong to an anonymous default package. Two classes with the same name can coexist if they reside in different packages, and must be referenced by their fully qualified names to avoid ambiguity:

```

1 casino.game.Deck cards = new casino.game.Deck();
2 star.cruise.Deck deck = new star.cruise.Deck();

```

3.5 Final Classes and Methods

Definition 3.5.1 (*final Method*). A method declared `final` cannot be overridden in any subclass.

Definition 3.5.2 (*final Class*). A class declared `final` cannot be subclassed. All its methods are implicitly `final`.

Using `final` provides two benefits:

1. **Security** ensures behaviour cannot be silently altered by a subclass.
2. **Efficiency** allows the compiler to resolve method calls at compile time (static binding) rather than at runtime (dynamic dispatch), which is discussed further in Chapter 5 on Polymorphism.

3.6 Abstract Classes and Methods

As a class hierarchy grows more general towards the root, a superclass may become too abstract to instantiate meaningfully. Such classes are called **abstract classes**.

Definition 3.6.1 (*Abstract Class*). An **abstract class** is declared with the `abstract` keyword and cannot be instantiated. It serves as a blueprint for subclasses.

Definition 3.6.2 (*Abstract Method*). An **abstract method** is declared with the `abstract` keyword and has no method body. Its implementation must be provided by every concrete subclass.

```
1 public abstract class Figure {  
2     private String color;  
3  
4     public Figure() { color = "black"; }  
5     public Figure(String c) { this.color = c; }  
6     public String getColor() { return color; }  
7  
8     // Abstract methods no implementation here  
9     public abstract double findArea();  
10    public abstract double findPerimeter();  
11    public abstract void print();  
12 }
```

Concrete subclasses such as `Rectangle` and `Circle` must provide implementations for all three abstract methods, or they too must be declared **abstract**.

In UML class diagrams, abstract class names and abstract method names are shown in *italics*, or annotated with the `{abstract}` constraint.

Warning: Cannot Instantiate Abstract Classes

Attempting `new Figure()` will cause a compile-time error. Abstract classes can only be used as reference types: `Figure f = new Circle(...)`; is legal because `Circle` is concrete.

3.7 Multiple Inheritance and Interfaces

3.7.1 Why Multiple Inheritance Is Needed

Real-world modelling sometimes demands that a class inherit from more than one parent. Examples: a `Library` is both a `Building` and a `Container`; a `PartTimeStudent` is both a `Student` and an `Employee`.

Java deliberately **does not support multiple class inheritance** to avoid ambiguity (the “diamond problem”). Instead, it supports **multiple interface implementation**.

3.7.2 Interfaces

Definition 3.7.1 (Interface). An **interface** is similar to an abstract class, but may only contain *abstract methods* and *constants* (`static final` fields). A class *implements* an interface and must provide concrete implementations for all its methods.

```
1 public interface IntFigure {
2     public abstract double findPerimeter();
3     public abstract double findArea();
4     public abstract void print();
5 }
6
7 public class Circle implements IntFigure {
8     private double radius;
9     public Circle(double rad) { this.radius = rad; }
10    public double findArea() { return Math.PI * radius * radius; }
11    public double findPerimeter() { return 2 * Math.PI * radius; }
12    public void print() { /* ... */ }
13 }
```

3.7.3 Derived Interfaces

An interface can extend another interface using the `extends` keyword. A concrete class implementing a derived interface must implement *all* methods from both the derived and base interfaces:

```
1 public interface IOrdered {
2     public boolean precedes(Object other);
3     public boolean follows(Object other);
4 }
5 public interface IShowablyOrdered extends IOrdered {
6     public void showOneWhoPrecedes();
7 }
8 public class Order implements IShowablyOrdered {
9     public boolean precedes(Object other) { /* ... */ }
10    public boolean follows(Object other) { /* ... */ }
11    public void showOneWhoPrecedes() { /* ... */ }
12 }
```

3.7.4 Abstract Classes vs. Interfaces

Feature	Abstract Class	Interface
Methods	May have concrete methods	Abstract methods only
Fields	Any fields allowed	Constants only (<code>static final</code>)
Constructor	Yes	No
Inheritance	Extend one class only	Implement many interfaces
Relationship	“is-a” (real parent)	“can-do” (type/contract)

Key Result: Abstract vs. Interface

Use an **abstract class** when subclasses share common state or non-trivial default behaviour. Use an **interface** to define a contract that unrelated classes can fulfil, enabling a form of multiple inheritance.

3.7.5 Multiple Inheritance via Interfaces in Code

A class may extend one superclass and implement multiple interfaces simultaneously:

```
1 public class SubClass1 extends SuperClass
2     implements Interface1, Interface2, Interface3 {
3     // must implement all methods from Interface1, Interface2, Interface3
4 }
```

Interfaces are especially useful for assigning common behaviour to otherwise unrelated classes. For example, an `INameable` interface (with a single `name()` method) could be implemented by `Person`, `Pet`, `Ship`, and `Building` classes that have no common parent.

Chapter 4

Exception Handling

Building on the class and inheritance concepts from Chapters 2.1 and 3.1, this chapter introduces **exception handling** — Java’s mechanism for detecting and recovering from runtime errors in a structured, object-oriented way.

4.0.1 Learning Objectives

By the end of this chapter, you should be able to:

- Handle errors in Java programs gracefully.
- Understand and apply Java’s exception handling mechanism.
- Summarise Java’s exception class hierarchy.
- Use predefined exception classes effectively.
- Create and throw your own custom exception classes.

4.1 Error Handling

Definition 4.1.1 (Exception). An **exception** is a runtime anomaly that a program may detect during execution. Examples include memory exhaustion, failure to open an input file, and division by zero. Such anomalies may disrupt normal program execution if not handled.

Well-designed programs include *error-handling code* to deal with these anomalies before they cause crashes or incorrect results. Java provides a built-in exception handling mechanism to catch and handle runtime exceptions. An **exception handler** (or recovery procedure) catches an exception and attempts to recover from the problem.

4.1.1 Naive Error Handling vs. Exception Handling

Consider a program that computes the average of student marks. Without any error handling (Version 1), dividing by zero when `numOfStudents = 0` causes undefined behaviour. A basic approach (Version 2) uses an `if` check to print an error and call `System.exit(0)`, but this mixes normal logic with error-handling code and terminates the program immediately.

Java’s exception mechanism (Version 3 onward) separates the normal code path from error handling, allowing the program to continue executing after an exception is caught:

```
1 try {
2     numOfStudents = sc.nextInt();
3     if (numOfStudents <= 0)
4         throw new Exception(
5             "Error: no of students must not equal to 0!");
6     // ... rest of normal logic ...
7     System.out.println("Average marks = " + avgMarks);
8 }
```

```

9  catch (Exception e) {
10     System.out.println(e.getMessage());
11 }
12 System.out.println("End of program execution!");

```

When `numOfStudents = 0`, the `throw` fires, execution jumps to the `catch` block, and the program then continues with the statement after the `catch` — printing "End of program execution!" in both the normal and exceptional cases.

4.2 Java's Exception Handling Mechanism

Java exception handling is built around three actions:

1. **Trying:** Enclose code that might throw an exception inside a `try` block.
2. **Throwing:** When an error condition is detected, create and `throw` an exception object containing the type of exception and the program state at the time of the error.
3. **Catching:** The JVM searches for a `catch` block whose parameter type matches the thrown exception type. The matching handler is executed; unmatched handlers are skipped.

4.2.1 The try/throw/catch/finally Syntax

```

1  try {
2      // normal flow; at least one statement may throw
3      if (/* error condition */)
4          throw new ExceptionClassName(optionalMessage);
5  }
6  catch (ExceptionClass1 e1) {
7      // handle exception type 1
8  }
9  // ... additional catch blocks ...
10 catch (ExceptionClassN eN) {
11     // handle exception type N
12 }
13 finally {
14     // (optional) always executed, whether or not an exception was thrown
15 }

```

The `finally` block is useful for cleanup operations (e.g. closing files or releasing resources) that must occur regardless of whether an exception was thrown.

Key Result: Flow of Control

No exception: All `try` statements execute; all `catch` blocks are skipped; `finally` runs; execution continues after the entire construct.

Exception thrown: Statements after the `throw` inside `try` are skipped; the first matching `catch` block executes; `finally` runs; execution continues after the construct.

4.3 Java's Exception Hierarchy

All exception and error types in Java form an inheritance hierarchy rooted at `Object`:

- `Object` → `Throwable`
 - `Exception` — recoverable conditions; what programs normally handle.

- * `ClassNotFoundException`, `IOException`, `AWTException`, ...
- * `RuntimeException` (and all its subclasses) — *unchecked*
 - `ArithmeticException`, `NullPointerException`,
 - `IndexOutOfBoundsException`,
 - `ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException`,
 - `ClassCastException`, `IllegalArgumentException`,
 - `NumberFormatException`, ...
- **Error** — serious JVM-level failures; not normally caught. *Unchecked*.
 - * `LinkageError`, `VirtualMachineError`, `AWTError`, ...

In the hierarchy, more *general* exception classes are higher up; more *specific* ones are lower. A catch block for a general type will also catch all its subtypes (IS-A relationship from Chapter 3.1).

4.3.1 Commonly Used Predefined Exceptions

Exception Class	Cause
<code>ArithmeticException</code>	Division by zero or similar arithmetic error
<code>IndexOutOfBoundsException</code>	Array or string index out of valid range
<code>ArrayIndexOutOfBoundsException</code>	Array index < 0 or $\geq$ array length
<code>StringIndexOutOfBoundsException</code>	String index < 0 or $\geq$ string length
<code>FileNotFoundException</code>	Named file cannot be found
<code>IllegalArgumentException</code>	Improper argument passed to a method
<code>NullPointerException</code>	Dereferencing an uninitialised object reference
<code>NumberFormatException</code>	String cannot be parsed as a number

Warning: Unchecked Exceptions Are Still Dangerous

Unchecked exceptions (subclasses of `RuntimeException` or **Error**) do not require explicit handling. If left uncaught, they propagate to Java's default handler, which terminates the program with an error message. Always guard against them through careful programming.

4.4 Using Exception Classes

4.4.1 The Exception Class

Definition 4.4.1 (Exception Constructors). `Exception` provides two constructors:

```

1 public Exception()
2 public Exception(String message)

```

Key instance methods inherited by all exception objects:

- `String getMessage()` — returns the message string stored in the exception.
- `String toString()` — returns a short description of the exception object.
- `void printStackTrace()` — prints the full call-stack trace leading to the throw point.

The `printStackTrace()` output names every method call from `main` down to the point where the exception was thrown, which is invaluable for debugging:

```

1 / by zero
2 java.lang.ArithmeticException: / by zero
3     at CreateException.method3(CreateException.java:36)
4     at CreateException.method2(CreateException.java:31)
5     at CreateException.method1(CreateException.java:28)
6     at CreateException.main(CreateException.java:16)

```

4.4.2 Checked vs. Unchecked Exceptions

Definition 4.4.2 (Checked Exception). A **checked exception** is one that the compiler can detect as potentially occurring. It must be either (a) caught within the method using `try/catch`, or (b) declared in the method signature with the `throws` clause so the caller handles it.

Definition 4.4.3 (Unchecked Exception). An **unchecked exception** belongs to `RuntimeException` or `Error`. The compiler does not force the programmer to handle it, though doing so is often good practice.

Key Result: Checked vs. Unchecked Rule

If an exception is a subclass of `RuntimeException` or `Error` $\Rightarrow$ **unchecked**: handling is optional.

Otherwise $\Rightarrow$ **checked**: must catch it *or* declare `throws` in the method signature.

4.4.3 Throwing Exceptions from Methods

When a method detects an error condition but does not handle it itself, it can propagate the exception to the caller using the `throws` clause:

```

1 public static double computeAvgMarks()
2     throws ArithmeticException {
3     // ...
4     if (numOfStudents <= 0)
5         throw new ArithmeticException(
6             "Error: no of students must not equal to 0!");
7     // ...
8     return avgMarks;
9 }

```

The calling method must then place the call inside a `try` block with a matching `catch`:

```

1 try {
2     average = computeAvgMarks();
3     System.out.println("Average marks = " + average);
4 }
5 catch (ArithmeticException e) {
6     System.out.println(e.getMessage());
7 }
8 finally {
9     System.out.println("End of program execution!");
10 }

```

4.4.4 Exception Propagation

If an exception is thrown inside a method and no matching `catch` block exists in that method, the exception **propagates** up the call stack to the invoking method. This continues until either a matching handler is found or the exception reaches the JVM's default handler.

Example 4.4.4 (Exception Propagation). Suppose `main` calls `method1`, which calls `method2`, which calls `method3`. If `method3` throws an `ArrayIndexOutOfBoundsException` and neither `method3` nor `method2` catches it, the exception propagates back to `method1` where a matching catch block handles it. Execution then resumes normally after `method1`'s `try/catch` construct.

4.4.5 Multiple catch Blocks

A single `try` block may be followed by several `catch` blocks. The JVM searches them *in order* and executes only the first matching one.

Exam Tip: Order Your catch Blocks Correctly

Always place **more specific** exception types *before* more general ones. Because `ArithmeticException` is a subtype of `RuntimeException`, which is a subtype of `Exception`, placing `catch (Exception e)` first would swallow every exception before the specific handlers ever run. The compiler will flag this as an error for checked exceptions; for unchecked ones it is a silent logical bug.

```
1 catch (ArithmeticException e) { // more specific -- first
2     System.out.println(e.getMessage());
3     System.exit(0);
4 }
5 catch (Exception e) {           // more general -- last
6     System.out.println(e.getMessage());
7 }
```

4.5 Creating Custom Exception Classes

Java allows you to define your own exception classes for application-specific error conditions. A custom exception must **extend** an existing exception class (direct or indirect).

Definition 4.5.1 (Custom Exception). A **custom exception class** is a user-defined class that inherits from an existing exception class. Typically only constructors are defined, each calling the parent's constructor via `super()`.

```
1 public class IntNonNegativeException extends Exception {
2
3     public IntNonNegativeException() {
4         super("Integer input is a negative number!!");
5     }
6
7     public IntNonNegativeException(String message) {
8         super(message);
9     }
10 }
```

The custom exception is thrown, caught, and used exactly like any predefined exception:

```
1 public int getInteger() {
2     int num = 0;
3     Scanner sc = new Scanner(System.in);
4     try {
5         System.out.print("Enter the integer: ");
6         num = sc.nextInt();
7         if (num < 0)
8             throw new IntNonNegativeException(); // default constructor
9     }
```

```
9     }
10     catch (IntNonNegativeException e) {
11         System.out.println(e.getMessage());
12         num = getIntAgain(); // second chance
13     }
14     return num;
15 }
```

Key Result: Custom Exception Guidelines

- Extend the most appropriate existing exception class (commonly `Exception` for checked, or `RuntimeException` for unchecked).
- Provide a no-argument constructor with a sensible default message via `super("...")`.
- Provide a `String` message constructor that forwards to `super(message)`.
- Name the class to clearly describe the error condition (`XxxException`).

Chapter 5

Polymorphism

Building on the inheritance hierarchy established in Chapter 3.1 and the abstract class / interface machinery from Sections 3.6 and 3.7.2, this chapter explores **polymorphism** — the third pillar of object-oriented programming. Where Chapter 1 introduced the concept informally (see Definition 1.5.17), here we develop it rigorously: how method dispatch works at runtime, the rules governing type casts, and the concrete benefits polymorphism brings to software design.

5.0.1 Learning Objectives

After completing this chapter, you should be able to:

- Explain what polymorphism means in the context of OOP.
- Distinguish between static binding (compile time) and dynamic binding (runtime).
- Describe upcasting and downcasting, and apply `instanceof` safely.
- Articulate the benefits of polymorphism: simplicity and extensibility.
- Identify the three forms of method overriding.

5.1 Polymorphism

Definition 5.1.1 (Polymorphism). The word *polymorphism* derives from Greek, meaning “*many forms*.” In OOP, it refers to the ability of an object reference to point to objects of different types; which method body is actually executed depends on the *runtime type* of the object, not the declared type of the reference.

Polymorphism is made possible through **method overriding**: a subclass redefines a method inherited from a parent class using *exactly the same signature and return type*. The subclass version may either *replace* the parent’s behaviour entirely, or *refine* it by delegating part of the work back to the parent via `super`.

5.1.1 Method Overriding: Replace vs. Refine

Consider a `Person` superclass with a `printInfo()` method. An `Employee` subclass can override this method in two ways:

5.1: Replace: `printInfo()` is completely rewritten in `Employee`.

```
1 public class Employee extends Person {
2     double salary;
3     // ... constructor, accessors ...
4
5     // REPLACE: entirely new implementation --- does not call super
6     public void printInfo() {
7         System.out.println("Employee name is: " + getName())
```

```

8         + " and age is " + getAge());
9     System.out.println("Salary is " + salary);
10 }
11 }

```

5.2: Refine: `printInfo()` in `Employee` calls `super` and adds extra output.

```

1 public class Employee extends Person {
2     double salary;
3
4     // REFINE: extends the parent's implementation
5     public void printInfo() {
6         System.out.println("Employee details are:");
7         super.printInfo(); // reuse Person's version
8         System.out.println("Salary is " + salary);
9     }
10 }

```

5.1.2 Classic Polymorphism Pattern

A canonical use of polymorphism is writing a method that accepts a *superclass reference* and dispatches to the correct subclass version automatically:

5.3: A single `call()` method handles any `Animal` subclass.

```

1 void call(Animal animal) {
2     animal.move(); // dispatches to Eagle.move(), Tiger.move(), etc.
3 }
4
5 call(new Eagle()); // => Eagle's move()
6 call(new Tiger()); // => Tiger's move()
7 call(new Shark()); // => Shark's move()

```

The method `call()` never needs to be modified when new `Animal` subclasses are added — a key design advantage.

Key Result: Polymorphism

When a method is invoked through a superclass reference, Java resolves it to the *subclass override* based on the **actual runtime type** of the object. The same method name and signature can produce different behaviours depending on which concrete class is instantiated.

5.1.3 The Liskov Substitution Principle

Definition 5.1.2 (Liskov Substitution Principle (LSP)). Barbara Liskov (1988): if S is a subtype of T , then any program P written in terms of T should behave identically when objects of type S are substituted for objects of type T . In practice: *subtypes must be fully substitutable for their base types*.

LSP justifies why polymorphism works correctly — a `Eagle` reference held as an `Animal` must honour the entire `Animal` contract. Violating LSP (e.g. overriding a method to throw an exception the parent never throws) breaks code that relies on the polymorphic guarantee.

5.2 Binding

Definition 5.2.1 (Binding). *Binding* is the process of connecting a method call to the method body that will execute it. The moment at which this connection is established distinguishes two forms.

5.2.1 Static Binding (Early Binding)

Static binding resolves the method call at **compile time**, based on the *declared (reference) type*. It applies to:

- **private** methods (not visible to subclasses),
- **static** methods (belong to the class, not the object),
- **final** methods (cannot be overridden).

Static binding also governs **method overloading**: since the correct overload is chosen by matching the compile-time types of the arguments, the dispatch happens before the program runs.

5.4: Static binding example — the `Collection` overload is chosen at compile time because `c`'s declared type is `Collection`.

```
1 public class StaticBindingTest {
2     public static void main(String[] args) {
3         Collection c = new HashSet();    // declared type: Collection
4         StaticBindingTest et = new StaticBindingTest();
5         et.sort(c);                      // compiler picks sort(Collection)
6     }
7
8     public Collection sort(Collection c) {
9         System.out.println("Inside Collection sort method");
10        return c;
11    }
12
13    public Collection sort(HashSet hs) { // never called here
14        System.out.println("Inside HashSet sort method");
15        return hs;
16    }
17 }
```

5.2.2 Dynamic Binding (Late Binding)

Dynamic binding defers method resolution to **runtime**, based on the *actual (object) type*. Java uses dynamic binding for all non-private, non-static, non-final methods.

5.5: Dynamic binding — `vehicle.start()` dispatches to `Car.start()` at runtime.

```
1 class Vehicle {
2     public void start() {
3         System.out.println("Inside start method of Vehicle");
4     }
5 }
6
7 class Car extends Vehicle {
8     @Override
9     public void start() {
10        System.out.println("Inside start method of Car");
11    }
12 }
```

```

12 }
13
14 class Bus extends Vehicle {
15     @Override
16     public void start() {
17         System.out.println("Inside start method of Bus");
18     }
19 }
20
21 public class DynamicBindingTest {
22     public static void main(String[] args) {
23         Vehicle vehicle = new Car(); // reference type: Vehicle
24         vehicle.start();              // OUTPUT: Inside start method of Car
25     }
26 }

```

The reference type (`Vehicle`) governs what methods the compiler *allows* you to call; the object type (`Car`) governs which body runs.

Key Result: Dynamic Binding in Java

Java defaults to **dynamic binding** for instance methods. The method body executed is determined by the *runtime type* of the object, not by the declared type of the reference. This is the mechanism that makes polymorphism work.

Static vs. Dynamic Binding at a Glance

- **Method overloading** $\Rightarrow$ static binding (compile time, matches argument types).
- **Method overriding** $\Rightarrow$ dynamic binding (runtime, matches object type).
- `private` / `static` / `final` methods $\Rightarrow$ always static binding.

5.3 Object Typecasting

Polymorphism allows a superclass reference to point to a subclass object. Java provides explicit cast syntax to navigate this type hierarchy.

5.3.1 Upcasting

Definition 5.3.1 (Upcasting). Assigning a subclass object to a superclass (or ancestor) variable is called *upcasting*. It is always safe and may be done implicitly.

```

1 Shape shape;                // base class reference
2 Rectangle rect = new Rectangle(15, 10);
3 shape = rect;                // upcast --- implicit, always safe
4 System.out.println(shape.findArea()); // dynamic binding => Rectangle.findArea()

```

Even though `shape` is declared as a `Shape`, calling `findArea()` uses `Rectangle`'s implementation via dynamic binding.

5.3.2 Downcasting

Definition 5.3.2 (Downcasting). A type cast from a base class reference to a derived class type is called *downcasting*. It must be done explicitly and is *not* checked by the compiler; an incorrect downcast produces a `ClassCastException` at runtime.

```

1 Shape shape = new Shape();           // object type is Shape
2 Rectangle rect = shape;               // COMPILE ERROR --- cannot assign directly
3 Rectangle rect = (Rectangle) shape;   // explicit cast --- compile OK
4                                     // but RUNTIME ERROR (ClassCastException)!
5 int h = rect.getHeight();             // compile OK (compiler trusts the cast)

```

Downcasting Pitfall

The compiler accepts an explicit downcast without complaint, but if the actual runtime object is not of the target type a `ClassCastException` will be thrown. Always verify with `instanceof` before downcasting.

5.3.3 The instanceof Operator

Definition 5.3.3 (`instanceof`). The expression `object instanceof ClassName` returns `true` if `object` is an instance of `ClassName` or any of its descendant classes; it returns `false` otherwise (including when `object` is `null`).

A standard pattern for safe downcasting is:

5.6: Safe downcast using `instanceof`.

```

1 public boolean equals(Object anObject) {
2     if (anObject == null)
3         return false;
4     if (!(anObject instanceof Person))
5         return false;
6     Person aPerson = (Person) anObject; // safe downcast
7     return name.equals(aPerson.getName()) && (age == aPerson.getAge());
8 }

```

5.3.4 Reference Type vs. Object Type: Compile-Time and Runtime Checks

In the declaration `<ReferenceType> ref = new <ObjectType>();`:

- The **reference type** is checked at *compile time* — it determines which method names are accessible.
- The **object type** is checked at *runtime* — it determines which method body executes (dynamic binding) and whether a downcast is valid.

Assignment Compatibility Rules

1. Superclass ref $\leftarrow$ superclass object: always OK.
2. Subclass ref $\leftarrow$ subclass object: always OK.
3. Superclass ref $\leftarrow$ subclass object (*upcast*): always OK; subclass-only members inaccessible through the reference.
4. Subclass ref $\leftarrow$ superclass object: compile error without explicit cast; runtime error if the object is truly of the superclass type.

5.4 Benefits of Polymorphism

5.4.1 Simplicity

Polymorphism lets code interact with an entire family of types through a single base-type interface. Client code does not need to know which concrete subclass it is handling; it just invokes the method, and the correct version runs automatically. This reduces the cognitive load on readers and makes the code shorter.

5.4.2 Extensibility

Adding a new subclass requires *no changes* to existing code that uses the base type. A new `Parrot` class extending `Animal` automatically works with any method that accepts an `Animal` reference — no modifications to `call()`, no new overloads, no switch-statements.

5.7: Adding a new `Circle` subtype requires zero changes to the loop.

```
1 public abstract class Shape {
2     public abstract double area();
3 }
4
5 public class Circle extends Shape {
6     private double radius;
7     public Circle(double r) { radius = r; }
8     public double area() { return Math.PI * radius * radius; }
9 }
10
11 public class Rectangle extends Shape {
12     private double width, height;
13     public Rectangle(double w, double h) { width = w; height = h; }
14     public double area() { return width * height; }
15 }
16
17 // Client code --- unchanged as new Shape subclasses are added
18 for (Shape s : shapes) {
19     System.out.println("Area = " + s.area());
20 }
```

5.5 Three Ways of Method Overriding

The slides identify three distinct contexts in which a subclass can provide a method body that “overrides” an inherited specification:

1. **Classic overriding:** a concrete subclass overrides a concrete method inherited from a concrete superclass (e.g. `Employee.printInfo()` overriding `Person.printInfo()`).
2. **Implementing abstract methods:** a concrete subclass provides a body for an abstract method declared in an abstract superclass (e.g. `Dog.speak()` implementing `LivingThings.speak()`). Recall from Section 3.6 that a class with any unimplemented abstract method is itself abstract.
3. **Implementing interface methods:** a concrete class provides bodies for all methods declared in a Java interface (e.g. `Cat` and `Lion` implementing `IAnimal.move()` and `IAnimal.speak()`). This was introduced in Section 3.7.2.

All three forms participate in dynamic binding equally — the runtime type of the object determines which body runs, regardless of whether the origin was a class, abstract class, or interface.

5.6 Extended Examples

5.6.1 Interface-Based Polymorphism: IAnimal

5.8: Interface-based polymorphism. `CareTaker.takeAWalk()` accepts any `IAnimal` — dispatch is dynamic at runtime.

```
1 interface IAnimal {
2     void move();
3     void speak();
4 }
5
6 class Cat implements IAnimal {
7     public void move() { System.out.println("Cat moves..."); }
8     public void speak() { System.out.println("Meow !"); }
9 }
10
11 class Lion implements IAnimal {
12     public void move() { System.out.println("Lion moves..."); }
13     public void speak() { System.out.println("ROAR !"); }
14 }
15
16 class CareTaker {
17     public void takeAWalk(IAnimal pet) {
18         pet.move(); // dynamic binding --- actual type resolved at runtime
19         pet.speak();
20     }
21 }
22
23 class AnyClass {
24     public static void main(String[] args) {
25         CareTaker ct = new CareTaker();
26         ct.takeAWalk(new Cat()); // => Cat moves... / Meow !
27         ct.takeAWalk(new Lion()); // => Lion moves... / ROAR !
28     }
29 }
```

Compare this with method *overloading*: if `CareTaker` instead defined separate `takeAWalk(Cat cat)`, `takeAWalk(Lion lion)`, etc., the compiler would resolve the call at compile time (static binding). Adding a new animal type would require adding yet another overload. The interface + polymorphism approach requires *zero* changes to `CareTaker`.

5.6.2 Abstract Class Hierarchy: LivingThings

5.9: Abstract class hierarchy. `Mammal` is abstract because it does not implement `speak()`; `Dog` is concrete.

```
1 abstract class LivingThings {
2     public void grow() { System.out.println("LivingThings grow!"); }
3     public abstract void speak();
4 }
5
6 abstract class Mammal extends LivingThings {
7     public void move() { System.out.println("Mammal move!"); }
8     public void grow() { System.out.println("Mammal grow!"); } // override
9     public void eat() { System.out.println("Mammal eat!"); }
10    // speak() still abstract --- Mammal does not implement it
11 }
12
13 class Dog extends Mammal { // concrete --- must implement speak()
14 }
```

```

14 public void speak() { System.out.println("Woof!"); }
15 public void move() { System.out.println("Dog move!"); } // override
16 public void grow() { System.out.println("Dog grow!"); } // override
17 // eat() inherited from Mammal
18 }

```

Running:

```

1 Mammal m = new Dog();
2 m.speak(); // => Woof! (Dog.speak --- implements abstract)
3 m.move(); // => Dog move! (Dog.move --- overrides Mammal.move)
4 m.eat(); // => Mammal eat! (inherited --- Dog has no eat())
5 m.grow(); // => Dog grow! (Dog.grow --- overrides Mammal.grow)

```

Note that the output is identical whether the reference type is `Dog` or `Mammal`, because dynamic binding always dispatches to the *actual* type (`Dog`).

Chapter 6

UML Class Diagrams and Class Relationships

6.1 Why Model?

Before writing code for a complex system, it helps to create an abstract representation of it — a *model*. Models serve several purposes:

- They simplify reality to a level of detail that is tractable (neither too abstract nor too concrete).
- They aid visualisation, communication, and understanding among team members.
- They act as blueprints for design creation and investigation.
- They serve as living documentation of the system.

A good model sits at the right level of abstraction — like an engineering blueprint that is detailed enough to build from, but not cluttered with every manufacturing constraint. As Einstein said: *“Everything should be made as simple as possible, but not simpler.”*

6.2 The Unified Modeling Language (UML)

Definition 6.2.1 (Unified Modeling Language). The **Unified Modeling Language (UML)** is a standardised visual language for specifying, documenting, and communicating various aspects of complex software systems.

UML was created by Booch, Jacobson, and Rumbaugh in 1996 (the “Three Amigos”), and version 1.1 was adopted by the Object Management Group (OMG) in 1997. The official specification is maintained at <http://www.omg.org/spec/UML/>.

UML is *not* a programming language — it is a family of diagram types that collectively describe both the **structure** and the **behaviour** of a system.

6.2.1 UML Diagram Types

UML diagrams are divided into two top-level categories:

1. **Structure Diagrams** — describe the static architecture of a system:

- **Class Diagram** (the focus of this chapter)
- Object Diagram
- Component Diagram
- Composite Structure Diagram
- Deployment Diagram
- Package Diagram

2. **Behaviour Diagrams** — describe dynamic aspects:

- Activity Diagram
- Use Case Diagram
- State Machine Diagram
- *Interaction Diagrams* (Sequence, Communication, Timing, Interaction Overview)

Key Result: UML Diagram Taxonomy

UML diagrams split into **Structure** (what the system *is*) and **Behaviour** (what the system *does*). The Class Diagram is the most important structure diagram and forms the backbone of OO design.

6.3 Class Diagrams

A **class diagram** is a structure diagram that shows the classes in a system together with their attributes, operations, and relationships.

6.3.1 Class Notation

A class is drawn as a rectangle divided into three compartments:

1. **Name compartment** — the class name (bold, centred).
2. **Attribute compartment** — list of attributes.
3. **Operation compartment** — list of operations (methods).

6.3.1.1 Attribute Specification

The full format for an attribute is:

visibility name : type [multiplicity] = initial-value

- **Visibility:** + public, - private, # protected, ~ package (default).
- **Multiplicity:** e.g. [1..*] means one or more values.
- **Static** members are underlined.

Example (the **Order** class):

6.1: Order class attributes

```

1 - dateReceived : Date
2 - isPrepaid    : Boolean = FALSE
3 - referenceNumber : String
4 - price        : Float
5 - dateModified : Date [1..*]
```

6.3.1.2 Operation Specification

The full format for an operation is:

visibility name(parameter-list) : return-type

where each parameter follows:

direction name : type [= default]

Direction keywords: `in` (input), `out` (output), `inout` (both).

Example:

```
1 + dispatch(in dateDispatched : Date) : Boolean
2 + close()
```

Exam Tip: Visibility Symbols

Memorise the four visibility symbols: `+` public, `-` private, `#` protected, `~` package. Static members are underlined in the diagram.

6.4 Class Relationships

UML defines a hierarchy of class relationships. Understanding which relationship to use, and how to translate it to Java, is a core exam skill.

Dependency $\supset$ Association $\supset$ {Aggregation, Composition}
Generalisation, Realisation

6.4.1 Dependency

Definition 6.4.1 (Dependency). A **dependency** (dashed arrow `--->`) indicates that a change in the *supplier* class may affect the *client* class. It is the weakest relationship.

In Java, a dependency arises when `ClassB` appears in `ClassA` only transiently — not stored as a field:

6.2: Dependency in Java

```
1 class ClassA {
2     // ClassB used as method return type
3     ClassB doSomething() { ... }
4
5     // ClassB used as method parameter type
6     void doOther(ClassB b) { ... }
7
8     // ClassB used as local variable
9     void doMore() {
10         ClassB b = new ClassB();
11         ...
12     }
13 }
```

Common dependency stereotypes include `<<use>>`, `<<call>>`, and `<<instantiate>>`.

Warning: Dependency vs. Association

A **dependency** (dashed line) means the relationship is transient — `ClassB` is used only inside a method, *not* stored as a field. An **association** (solid line) means `ClassB` is referenced persistently as an attribute. Confusing the two is a common error.

6.4.2 Association

Definition 6.4.2 (Association). An **association** (solid line, optionally with an arrow) indicates a persistent, structural relationship between classes. One class holds a reference to the other as an attribute.

Key annotations that can appear on an association:

Annotation	Meaning
Multiplicity	How many instances participate (e.g. 1, 0..1, 1..*, *)
Role name	The name of the reference as seen from the other end
Association name	Label on the line (often with a direction arrow)
Navigability	Open arrowhead = navigable; x = not navigable
Constraint	{ordered}, {bag}, {seq} etc.

6.4.2.1 Uni-directional vs. Bi-directional Association

A **uni-directional** association (arrow on one end) is implemented by storing one reference in only one class:

6.3: Uni-directional association

```
1 class ClassA {  
2     ClassB myRole; // ClassA knows about ClassB; ClassB does not know ClassA  
3 }
```

A **bi-directional** association (arrows on both ends, or no arrowheads) requires both classes to hold references to each other:

6.4: Bi-directional association

```
1 class ClassA {  
2     ClassB[] bObjs = new ClassB[5];  
3     // initialise: bObjs[0] = new ClassB(this);  
4 }  
5 class ClassB {  
6     ClassA[] aObjs = new ClassA[10];  
7     public ClassB(ClassA a) { aObjs[0] = a; }  
8 }
```

6.4.2.2 Reflexive Association

A class can be associated with itself (e.g. a **Person** can manage other **Person** objects). This is drawn as a loop from the class back to itself.

6.4.2.3 Association Classes

When an association itself has attributes (e.g. the *date* on which a **Borrower** loans a piece of **Equipment**), an **association class** is attached to the association line with a dashed line. It can equivalently be refactored into a separate class that holds references to both ends:

6.5: Association class refactored to separate class

```
1 // Original: Borrower *---- Loan ----1..* Equipment  
2 class Loan {  
3     private Date date;  
4     private Borrower borrower; // 1  
5     private Equipment equipment; // 1  
6 }
```

Key Result: Multiplicity Notation

1	Exactly one
0..1	Zero or one
1..*	One or more
*	Zero or more (any number)

6.4.3 Aggregation and Composition

Both aggregation and composition are specialised associations that express a *whole-part* (“has a”) relationship.

6.4.3.1 Aggregation (Weak)

Definition 6.4.3 (Aggregation). **Aggregation** (hollow diamond at the whole end, $\diamond$) is a *weak* whole-part relationship. The parts can exist independently of the whole. Deleting the whole does *not* automatically delete the parts.

Example: a `LectureGroup` aggregates `Person` objects. If the lecture group is disbanded, the persons still exist.

Java implementation — parts are added through a reference (they already exist elsewhere):

6.6: Aggregation in Java

```
1 class ClassA {
2     // Use ArrayList (or array) to hold references to existing ClassB objects
3     ArrayList<ClassB> parts = new ArrayList<>();
4
5     public void addPart(ClassB b) {
6         parts.add(b); // b was created outside; added by reference
7     }
8 }
```

6.4.3.2 Composition (Strong)

Definition 6.4.4 (Composition). **Composition** (filled diamond at the whole end, $\blacklozenge$) is a *strong* whole-part relationship. The whole **creates and owns** the parts. A part belongs to *at most one* composite at a time, and when the composite is destroyed, all its parts are destroyed with it.

Example: a `Human` is composed of `Arms` and `Legs`. Arms and legs cannot meaningfully exist without the human body that created them.

Java implementation — parts are created inside the whole’s constructor:

6.7: Composition in Java

```
1 class ClassA {
2     ArrayList<ClassB> parts = new ArrayList<>();
3
4     public ClassA() {
5         ClassB b1 = new ClassB(); // created BY ClassA
6         ClassB b2 = new ClassB();
7         parts.add(b1);
8         parts.add(b2);
9     }
10 }
```

Warning: Aggregation vs. Composition

The key distinction is **ownership and lifetime**:

- **Aggregation** (◇): parts exist independently; added by reference.
- **Composition** (◆): whole creates parts in its constructor; parts cannot exist without the whole.

In Java, both look similar (an `ArrayList` of parts), but composition creates parts *internally*, while aggregation accepts them *externally*.

6.4.4 Generalisation (Inheritance)

Definition 6.4.5 (Generalisation). **Generalisation** (solid line with hollow triangular arrowhead pointing to the parent) represents an “is a” relationship. The child class *inherits* all features of the parent and may specialise them.

In Java, generalisation maps directly to `extends`:

```
1 class ClassB extends ClassA {  
2     // ClassB inherits all of ClassA's features  
3 }
```

Example from the slides: `Shape ← Rectangle ← Square`. Square “is a” Rectangle “is a” Shape.

Generalisation hierarchies must be *directed* and *acyclical* — a class cannot be its own ancestor.

6.4.5 Realisation (Interface Implementation)

Definition 6.4.6 (Realisation). **Realisation** (dashed line with hollow triangular arrowhead) indicates that a class *implements* an interface, committing to fulfil all the operations declared by that interface.

In Java:

```
1 public interface IInterface {  
2     void someMethod();  
3 }  
4  
5 class ClassB implements IInterface {  
6     public void someMethod() { ... }  
7 }
```

Example 6.4.7 (Timer–Observer). The following diagram shows `Timer` depending on the <<interface>> `Observer`, while `TimeObserver` realises it:

```
1 public interface Observer {  
2     public void update(Object arg);  
3 }  
4  
5 class Timer {  
6     public void notify(Observer o) {  
7         o.update(this);  
8     }  
9 }  
10  
11 class TimeObserver implements Observer {
```

```

12 public void update(Object arg) {
13     // react to timer event
14 }
15 }

```

Notice that `Timer` depends on `Observer` (dashed arrow), while `TimeObserver` realises `Observer` (dashed hollow triangle). This is the classic **Observer** design pattern.

Key Result: Relationship Summary

Relationship	UML notation	Java keyword
Dependency	- - - -> (dashed arrow)	method param / local / return
Association	-----> (solid arrow)	instance field
Aggregation	---◇--- (hollow ◇)	field, added externally
Composition	---◆--- (filled ◆)	field, created internally
Generalisation	---▷ (solid hollow △)	extends
Realisation	- - -▷ (dashed hollow △)	implements

6.5 Abstract Classes and Interfaces in Class Diagrams

An **abstract class** is shown with the constraint `{abstract}` in the name compartment (or the class name in italics). Abstract operations are also marked `{abstract}`.

An **interface** is shown with the stereotype `<<interface>>` above the class name. All operations in an interface are implicitly abstract.

Example:

```

1 // UML: Shape {abstract} with areaOf() : float {abstract}
2 abstract class Shape {
3     public abstract float areaOf();
4 }
5
6 class Circle extends Shape {
7     public float areaOf() { return (float)(Math.PI * r * r); }
8 }

```

The Java Collections Framework is a practical example of this: `<<interface>> Collection` ← `<<interface>> List` ← `AbstractList {abstract}` ← `ArrayList`. An `Order` class holds a `List` reference (dependency on the interface), and at runtime an `ArrayList` is used — a classic example of *programming to an interface*.

6.6 Object Diagrams

Definition 6.6.1 (Object Diagram). An **object diagram** is a snapshot of the system at a particular instant in time, showing specific *instances* of classes and the links between them. It is a concrete, example-level counterpart of the abstract class diagram.

Notation differences from a class diagram:

- The name compartment is underlined: `objectName : ClassName`.
- An object without a name is an **anonymous object**: `: ClassName`.
- Attribute values (not types) are shown in the attribute compartment.
- Relationships between instances are called **links** (instances of associations).

Example: `b : Book [checked out] / title = "Twelfth Night" / author = "Shakespeare"`.

6.6.0.1 Anonymous Objects and Anonymous Classes

An anonymous object has no name:

```
1 library.checkout( new Book("Twelfth Night") );
2 // The Book object has no variable name -- anonymous object
```

An anonymous class is an unnamed class defined and instantiated in one expression, often used with interfaces:

```
1 Thread t = new Thread() {
2     public void run() {
3         // do something
4     }
5 };
6 t.run();
7 // Syntax: new InterfaceOrClassName() { class-body }
```

6.7 More about Class Diagram Notation

6.7.1 Visibility and Static Members

Symbol	Visibility
+	public
-	private
#	protected
~	package (default)
<u>underline</u>	static (class-level) member

6.7.2 Enumerations

An enumeration type is shown with the stereotype `<<enumeration>>`. The literal values appear in the attribute compartment. An instance of an enumeration shows the current literal in square brackets, e.g. `:DayOfWeek [Sunday]`.

6.7.3 Constraints

A **constraint** is a condition on an element, written in curly braces (natural language or OCL):

- {ordered} on a multi-valued association end — elements are in a defined order.
- {xor} between two associations from the same class — exactly one is active at a time.
- Attribute constraint example: `size : Integer {size >= 0}`.

6.8 Class Stereotypes

In OO design, classes are often categorised by their *responsibility* using three standard stereotypes. This classification guides how a system is decomposed:

Definition 6.8.1 (Entity Class). An **Entity** class (symbol: circle with a line underneath $\ominus$) holds **persistent information** tracked by the system. Entity classes correspond to the system's data model.

Naming convention: plain domain nouns — **Student**, **Course**, **Group**.

Definition 6.8.2 (Boundary Class). A **Boundary** class (symbol: circle with a line on the left $\vdash\bigcirc$) handles **interaction between the system and its environment** (users, external systems, hardware). Boundary classes implement the UI or system interfaces.

Naming convention: `xxxUI`, `xxxForm`, `xxxInterface` — e.g. `InvoiceForm`.

Definition 6.8.3 (Control Class). A **Control** class (symbol: circle with an arrow $\odot$) contains the **logic that coordinates and realises a use case**. Control classes orchestrate the interaction between boundary and entity classes.

Naming convention: `xxxMgr`, `xxxCtrl`, `xxxController` — e.g. `CourseMgr`.

Key Result: Entity–Boundary–Control (EBC) Pattern

In a well-structured OO system:

- **Boundary** classes handle I/O and call Control classes.
- **Control** classes contain the logic and manipulate Entity classes.
- **Entity** classes hold data — they do not know about the UI.

Example: `BenefitsForm` (Boundary) $\xrightarrow{\text{ijcall}}$ `EmployeeManager` (Control) $\xrightarrow{\text{ijuse}}$ `Employee`, `Benefits` (Entity).

Exam Tip: Using Stereotypes in Design Questions

When asked to design a system, identify the three stereotypes for each class before drawing the diagram. The stereotype determines naming conventions, what relationships are appropriate, and which classes should be *clients* vs. *suppliers*.

Chapter 7

UML Sequence Diagrams

7.1 Introduction

A Sequence Diagram is a type of UML Interaction diagram that models how objects collaborate to carry out a use case or operation. It focuses on the *sequence of messages* exchanged between participants over time, represented on vertical lifelines read top-to-bottom. Where a Class Diagram (Chapter 6) captures static structure, a Sequence Diagram captures dynamic behaviour.

7.1.1 Learning Objectives

- Describe the core elements of a Sequence Diagram.
- Identify and apply interaction fragments (`opt`, `alt`, `loop`, `break`, `ref`).
- Translate a Sequence Diagram into compilable Java code.

7.2 Sequence Diagram Elements

7.2.1 The Frame

Every Sequence Diagram is enclosed in a rectangular **frame**. A pentagon in the upper-left corner contains the keyword `sd` followed by the interaction name and optional parameters, for example `sd ButtonPress` or `sd printCurrentRecords(f : File)`.

7.2.2 Lifelines

Definition 7.2.1 (Lifeline). A **lifeline** represents a single participant (object, actor, or component) in an interaction. It consists of a head rectangle (or actor icon) labelled `[name] : ClassName` (the class name is mandatory; the instance name is optional) followed by a vertical dashed line stretching downward for the duration of the interaction.

In the `sd ButtonPress` example the participants are `:User`, `:Button`, `:ButtonListener`, and `:Light`.

7.2.3 Messages

Messages are drawn as horizontal arrows between lifelines (or looping back to the same lifeline). The four main variants are summarised in Table 7.1.

Message labels use the syntax:

- Simple call: `methodName(arg1, arg2)`
- Return with assignment: `result = dispatch(today)`
- Named parameter: `dispatch(dateDispatched = today)`

Type	Arrow	Meaning
Synchronous call	Solid line, filled arrowhead	Caller blocks until callee returns
Asynchronous call	Solid line, open arrowhead	Caller continues immediately
Return message	Dashed line, open arrowhead	Returns a value to the caller
Object creation	Dashed line, open arrowhead	Creates a new instance (new)

Table 7.1: Message variants in Sequence Diagrams.

- Undefined argument: `mymessage(14, -, 3.14)` (dash means the second argument is deliberately left unspecified)

7.2.4 Activation Boxes (Execution Specifications)

Definition 7.2.2 (Activation Box). A narrow rectangle overlaid on a lifeline is called an **activation box** (formally an *ExecutionSpecification*). It marks the period during which the corresponding object is actively executing a method. The box begins at the message arrival and ends at the return.

Nested activation boxes on the same lifeline indicate recursive or re-entrant calls (a method invoking itself or calling another method on the same object).

7.2.5 Self-Messages and Recursive Messages

A **self-message** is an arrow that loops from a lifeline back to itself, representing an object calling one of its own methods. A **recursive message** is a specific self-message whose activation box is nested inside the caller's own activation box, indicating a recursive method call. In Java this corresponds directly to a `this.methodName(...)` call or simple `methodName(...)` from within the same class.

Exam Tip: Identifying Self-Messages

If an arrow begins and ends on the *same* lifeline it is a self-message. If the receiving activation box is nested inside the sending one it is additionally recursive. Both produce a local method call in code — no cross-object reference is needed.

7.2.6 Object Creation and Destruction

When a participant is created *during* an interaction (rather than existing beforehand), it is shown with its lifeline head placed at the *y*-level where creation occurs, and a dashed creation message labelled **new** (or the constructor name) points to it. In Java this corresponds to a `new ClassName(...)` expression.

Object destruction is marked by an **X** at the end of the lifeline, indicating that the object's lifetime ends at that point (e.g. explicit null-ing or garbage collection in Java).

Example 7.2.3 (UpdateUserInfo — creation and destruction). In the `sd UpdateUserInfo` diagram, `:User` sends `updateUserInfo` to `:UserInfoForm`, which calls `:UserMgr`, which in turn creates a **new** `:UserInfo` object, calls `validate` and `update` on it, and then destroys it (X).

7.2.7 Message Specification and Parameter Modes

UML supports typed parameter annotations on messages, mirroring Java method signatures. An **in** parameter passes a value into the operation; **inout** passes a value in and receives a (possibly modified) value back; **out** carries a value only on return.

Example 7.2.4 (dispatch and verifyTransactions). • `dispatch(in dateDispatched : Date) : Boolean` — the caller passes `today` as the argument and captures the return in `result`: written `result = dispatch(today)`.

- `verifyTransactions(inout records : List<Transaction>)` — the caller passes its local list `myRecords` and receives the (potentially modified) list back: written `verifyTransactions(myRecords = records)`.

7.2.8 Found and Lost Messages

Definition 7.2.5 (Found Message). A **found message** has a known *receiving* event but no known sending event. It represents input arriving from outside the scope of the diagram (e.g. an external trigger or noise). Notation: a filled black circle at the *tail* of the arrow.

Definition 7.2.6 (Lost Message). A **lost message** has a known *sending* event but no receiving event — the message is interpreted as never reaching its destination. Notation: a filled black circle at the *head* of the arrow.

In the `sd printCurrentRecords` diagram the initial call `printCurrentRecords(f)` arrives as a found message (black dot at the arrow tail pointing to `:PrintManager`).

7.3 Interaction Fragments (Combined Fragments)

An **interaction fragment** is a named region drawn as a rectangle with a *label pentagon* in the upper-left corner. It constrains or repeats the messages inside it. The full set of operators is given in Table 7.2; the most exam-relevant ones are detailed below.

Operator	Java equivalent	Description
<code>opt</code>	<code>if (...) { }</code>	Single operand; executes only when guard is true.
<code>alt</code>	<code>if / else if / else</code> or <code>switch</code>	Multiple operands separated by a dashed line; exactly one is chosen based on its guard.
<code>loop</code>	<code>while / for</code>	Repeats while the guard expression is true.
<code>break</code>	<code>break</code> (exception)	Executes instead of the enclosing fragment when its guard is true; aborts the enclosing interaction.
<code>par</code>	Multi-threading	Operands execute concurrently.
<code>critical</code>	<code>synchronized</code> block	Operand cannot be interleaved with other occurrences on covered lifelines.
<code>ref</code>	Method call (reuse)	References another named Sequence Diagram (<i>InteractionUse</i>).

Table 7.2: Combined fragment operators.

7.3.1 `opt` — Optional Fragment

`opt` wraps a single block of messages that should only execute when its guard condition evaluates to true. It is equivalent to an `alt` with only one operand.

7.1: opt translated to Java

Example 7.3.1 (sd UserAccess — opt).

```
1 // opt [isAuthenticated]
2 if (isAuthenticated) {
3     msg("Please Enter");
4 }
```

7.3.2 alt — Alternative Fragment

alt contains two or more operands divided by dashed horizontal lines. Each operand carries a guard in square brackets; [else] is the default (at most one guard is true). This maps to if-else or switch in Java.

7.2: alt translated to Java

Example 7.3.2 (sd printCurrentRecords — alt).

```
1 // alt [rec == valid] / [else]
2 if (rec == valid) {
3     printerInterface.print(rec);
4 } else {
5     log(rec);
6 }
```

7.3.3 loop — Loop Fragment

The loop fragment repeats its body while the guard evaluates to true. Minimum and maximum iteration counts may appear in parentheses, e.g. loop(1,10).

7.3: loop with nested break translated to Java

Example 7.3.3 (sd printCurrentRecords — loop + break).

```
1 // loop [not end of file]
2 while (!endOfFile) {
3     rec = f.readRecord();
4     // inner alt for print / log ...
5
6     // break [y > 0] (nested inside a loop)
7     if (y > 0) {
8         save();
9         break;           // exits the loop entirely
10    }
11 }
```

Key Result: Fragment Operator Summary

- opt [cond] $\Rightarrow$ if (cond) {}
- alt [A] / [else] $\Rightarrow$ if (A) {} else {}
- loop [cond] $\Rightarrow$ while (cond) {}
- break [cond] $\Rightarrow$ if (cond) { ...; break; }
- ref N $\Rightarrow$ call to the interaction named N

7.3.4 ref — Interaction Use

A **ref** fragment (InteractionUse) references another complete Sequence Diagram by name. It allows shared sub-interactions to be factored out and reused, analogous to extracting a method. The referenced diagram is drawn separately and invoked wherever the **ref** box appears.

Example 7.3.4 (sd `UserAccess` — ref). The diagram contains two **ref** boxes: **ref** `EstablishAccess("Illegal PIN")` and **ref** `OpenDoor`. These represent separately-defined interactions that are composed into the main flow without duplicating their internal detail.

7.4 Diagram to Code

Translating a Sequence Diagram to Java follows a systematic correspondence between diagram elements and code constructs, summarised in Table 7.3.

Diagram element	Java construct
Lifeline <code>:ClassName</code>	Instance of class <code>ClassName</code> ; field or local variable
Message <code>m(args)</code> sent to lifeline	Method call <code>ref.m(args)</code>
Return message <code>v = m(...)</code>	Type <code>v = ref.m(...)</code> ;
Self-message <code>m(...)</code>	<code>this.m(...)</code> or plain <code>m(...)</code>
Object creation <code>new :C</code>	<code>C obj = new C(...)</code> ;
Object destruction (X)	End of scope / <code>obj = null</code> ;
<code>opt [cond]</code>	<code>if (cond) { }</code>
<code>alt [A] / [else B]</code>	<code>if (A) { } else { }</code>
<code>loop [cond]</code>	<code>while (cond) { }</code>

Table 7.3: Sequence Diagram element to Java code correspondence.

7.4.1 Worked Example: CalculatePrice

The sd `CalculatePrice` diagram involves four lifelines: `:Order`, `ol:OrderLine`, `p:Product`, and `c:Customer`. The interaction is initiated by a found message `calculatePrice()` to `:Order`, which then orchestrates the following steps:

1. Calls `ol.getQuantity()` → stores result in `qty`.
2. Calls `ol.getProduct()` → stores result in `p`.
3. Calls `p.getPricingDetails()` → stores result in `price`.
4. Self-calls `calculateBasicPrice(qty, price)` → stores in `bPrice`.
5. Self-calls `calculateDiscounts()` → stores in `discount`; internally this calls `c.getDiscountInfo()`.
6. Self-calls `calculateTotalPrice(price, discount)` → returns `total`.

The resulting Java classes are shown below. Note that even partial code must be **compilable** — stub return values are acceptable.

7.4: `Order.java` — translated from sd `CalculatePrice`

```
1 public class Order {
2     private Customer c;
3     private OrderLine ol; // other fields omitted
```

```

4
5     public double calculatePrice() {
6         int qty          = ol.getQuantity();
7         Product p        = ol.getProduct();
8         double price     = p.getPricingDetails();
9         double bPrice    = calculateBasicPrice(qty, price);
10        double discount   = calculateDiscounts();
11        double total      = calculateTotalPrice(price, discount);
12        return total;
13    }
14
15    private double calculateBasicPrice(int qty, double price) {
16        return qty * price;
17    }
18
19    private double calculateDiscounts() {
20        return c.getDiscountInfo();
21    }
22
23    private double calculateTotalPrice(double price, double discount) {
24        return price * discount;
25    }
26 }

```

7.5: OrderLine.java

```

1 public class OrderLine {
2     private int qty;
3     private Product product; // other fields omitted
4
5     public int getQuantity() {
6         return qty; // assume qty is initialised elsewhere
7     }
8
9     public Product getProduct() {
10        return product; // assume product is initialised elsewhere
11    }
12 }

```

7.6: Product.java and Customer.java (stubs)

```

1 public class Product {
2     public double getPricingDetails() {
3         return 100.0; // stub: some valid value
4     }
5 }
6
7 public class Customer {
8     public double getDiscountInfo() {
9         return 10.0; // stub: some valid value
10    }
11 }

```

Warning: Partial Code Must Still Compile

The slide deck emphasises this explicitly: even if you only implement a subset of the interaction, every class and method must be syntactically correct and compilable. Use stub return values (e.g. `return 0;` or `return null;`) for any method body that is not yet fully implemented.

Exam Tip: Reading a Sequence Diagram Systematically

When translating a diagram to code, work top-to-bottom along the *initiating* lifeline's activation box. For each outgoing arrow: (1) identify the target lifeline's class, (2) write the corresponding method call, (3) if there is a return arrow, capture the return value in a local variable. Handle **opt**/**alt**/**loop** boxes as you reach them in time-order.

Chapter 8

Modelling OO Applications

8.1 Introduction

This chapter brings together the UML Class Diagrams (Chapter 6) and UML Sequence Diagrams (Chapter 7) developed in earlier chapters and shows how to use them jointly to model a complete object-oriented application. The approach mirrors the way people naturally think about real-world systems: identify the *things* (objects) and then model how those things *interact*.

8.1.1 Learning Objectives

By the end of this chapter, students should be able to:

- Use Class and Sequence diagrams together to model OO applications end-to-end.
- Explain the steps in creating a design case study to implement a Blackjack game in Java.
- Design a Java software application based on a natural-language requirements statement.

8.2 Modelling Real-World Systems

The primary goal of object-oriented modelling is to represent real-world systems in a way that mirrors how people naturally think about them. Rather than decomposing a problem into procedures and data structures, OO design centres on identifying self-contained *objects* that encapsulate both data and behaviour.

8.2.1 Identifying Objects from Requirements

A practical starting point is to analyse a requirements statement linguistically:

- **Nouns** → candidate classes (or attributes).
- **Verbs** → candidate methods (responsibilities).

Example 8.2.1 (Noun/Verb Analysis). Consider the sentence: “*A student registered courses.*” The nouns *student* and *course* are candidate entity classes, while *registered* is a candidate method (`register()`) on one of those classes.

8.2.2 Class Stereotypes

When identifying classes, three stereotypes are commonly used to categorise them:

Definition 8.2.2 (Entity Class). An **Entity** class models persistent information tracked by the system (e.g. a **Student** or **Course** record). Entity objects typically correspond to rows in a database or persistent domain objects.

Definition 8.2.3 (Boundary Class). A **Boundary** class models the interaction between an actor and the system (e.g. a login form or a display screen). Boundary classes isolate the user interface from the application logic.

Definition 8.2.4 (Control Class). A **Control** class encapsulates the logic needed to realise a use case (e.g. an application manager). Control classes coordinate between boundary and entity classes.

Key Result: OO Modelling Steps

The standard iterative steps for modelling an OO application are:

1. Identify classes (entity, boundary, control).
2. Add attributes and methods to each class.
3. Add class relationships and multiplicities.
4. Identify object behaviours and interactions through functional-flow Sequence Diagrams.
5. Enhance class details (discover more methods from the Sequence Diagram).
6. Implement the model in code.
7. Quality-test the application.

Steps 4–5 form a feedback loop: drawing the Sequence Diagram often reveals methods that were not visible in the initial Class Diagram.

8.3 Short Example: Professor–Student–Course

The following running example illustrates how a brief requirements passage is transformed into a Class Diagram and then a Sequence Diagram.

8.3.1 Requirements Passage

“A professor is a mentor to at least one student. He/she can check the course results of his/her mentees, which the students have registered.”

8.3.2 Step 1 – Class Diagram

Applying noun extraction yields four candidate entity classes: **Professor**, **Student**, **Course**, and **Result**. Key observations:

- A **Professor** (*mentor*) is associated with 1..* **Student** objects (*mentee*); each student is mentored by exactly one professor (1 on the professor side).
- A **Student** registers for 1..* **Course** objects, and each course can be registered by 1..* students.
- **Result** is an entity class with attributes `-grade:String` and `-semYr:String`, linked to **Student** (1..* results per student). It is also associated with **Course** (each result belongs to one course).
- The note “*as Student Record*” on the **Professor** side indicates that a professor accesses student results in the role of reading a student record.

Attributes extracted from the passage:

- **Professor**: attribute `-name:String`; method `+getMenteeResults():void`
- **Student**: attribute `-name:String`; method `+getResults():List<r>`
- **Course**: attribute `-courseName:String`; method `+getCourseName():String`
- **Result**: attributes `-grade:String`, `-semYr:String`; methods `+getCourse():Course`, `+getGrade():String`

8.3.3 Step 2 – Sequence Diagram: sd check mentee result

The interaction for the use case *check mentee result* involves six lifelines: `:SelectForm` (boundary), `AppMgr` (control), `mentor:Professor`, `mentee:Student`, `result:Result`, and `course:Course`.

The message flow proceeds as follows:

1. **Professor** → **:SelectForm**: `select` (actor triggers UI).
2. **:SelectForm** → **AppMgr**: `showMenteeResults()` (boundary delegates to control).
3. **AppMgr** → **mentor:Professor**: `getMentees()` (retrieves list of mentees).
4. **loop** [foreach student]:
 - (a) **AppMgr** → **mentee:Student**: `getResults()` (fetches all results for that student).
 - (b) **loop** [foreach result]:
 - i. **AppMgr** → **result:Result**: `getCourse()` → returns `course:Course`.
 - ii. **AppMgr** → **course:Course**: `getCourseName()`.
 - iii. **AppMgr** → **result:Result**: `getGrade()`.
5. **AppMgr** → **:SelectForm**: `output` (return message with formatted results).

Exam Tip: Discovering Methods from Sequence Diagrams

Every message arrow in the Sequence Diagram corresponds to a method that must exist on the target lifeline's class. Constructing the Sequence Diagram *after* the initial Class Diagram is standard practice because it reliably surfaces additional methods (e.g. `getMentees()`, `getResults()`, `getCourse()`) that were not obvious at the noun-extraction stage.

8.4 Case Study: Blackjack

This section works through a complete design case study — designing a Java software application for the card game *Blackjack* (also called Twenty-One). The case study is adapted from *The Object-Oriented Thought Process* by Matt Weisfeld.

8.4.1 Requirements Summary

The intended purpose of the application is to implement a game of Blackjack. The key requirements are:

- One or more individual players play against the dealer (house).

- Each player plays only against the dealer, not against other players.
- The goal is to draw cards from the deck until the sum of face values equals 21, or is as close to 21 as possible without exceeding it.
- If the sum exceeds 21, the player loses (“bust”).
- If the first two cards sum to 21, the player has *Blackjack*.

8.4.1.1 Dealer’s Responsibilities

The dealer must:

- Deal cards and present additional cards on request.
- Show all or part of a hand.
- Calculate the value of all or part of a hand.
- Determine the winner and start a new hand.

8.4.1.2 Player’s Responsibilities

The player must be able to:

- Request a card (hit or hold).
- Display the hand and calculate its face value.
- Determine the number of cards in the hand.
- Respond to the dealer’s request for a new card or to start a new game.

8.4.1.3 Card Face Values

- Each card has its own face value; suit is irrelevant in Blackjack.
- Aces count as 1 or 11.
- Jack, Queen, and King each count as 10.
- All other cards represent their numeric face value.

8.4.1.4 Winning and Drawing Rules

- If the player’s total is closer to 21 than the dealer’s, the player wins an amount equal to the bet.
- A Blackjack win pays 3:2 times the bet (assuming the dealer does not also have Blackjack).
- If both the player and dealer have identical scores of at least 17, it is a *draw* and the player retains the bet.

8.4.2 Step 1 — Identify Candidate Classes (Noun Extraction)

Reading the requirements and listing all nouns produces:

Game, Blackjack, Dealer, House, Players, Player, Cards, Card, Deck, Hand, Face value, Suit, Winner, Ace, Face card, King, Queen, Jack, Bet.

8.4.3 Step 2 — Process of Elimination

Not every noun becomes a class. The following reasoning reduces the list:

- **Blackjack** → eliminated; it is the game/system itself, represented by the **Game** context.
- **House** → eliminated; it is just another name for **Dealer**.
- **Players** → eliminated; use a single **Player** class (objects are instantiated as needed).
- **Cards** → eliminated; use a single **Card** class.
- **Face value** → eliminated; it is an *attribute* of **Card** (`-value:int`).
- **Suit** → eliminated for this game (suits play no role in Blackjack), though it is kept as an attribute for potential future reuse.
- **Winner** → eliminated; *winner determination* is a *responsibility* of **Dealer**, not a separate class.
- **Ace, Face card, King, Queen, Jack** → all eliminated as classes; they are better represented as attribute values (e.g. `name`) within **Card**, with special-case logic handled by `getValue()`.
- **Bet** → retained tentatively; it carries non-trivial state (type, amount, coverage check) and may be promoted to a full class.

Key Result: Final Class List

After elimination, the five core classes are: **Card**, **Deck**, **Hand**, **Player**, **Dealer**. The **Bet** class is deferred but may be added later without redesigning the core.

Warning: CRC Cards Will Change

The first draft of CRC cards is expected to be imperfect. Not all initially proposed classes will survive to the final design, and new responsibilities will emerge as the Sequence Diagram is drawn. Treat the first draft as a starting point, not a contract.

8.4.4 Step 3 — Identify Class Responsibilities (Verb Extraction)

Responsibilities correspond to actions (verbs) extracted from the requirements.

8.4.4.1 Card

- **Know its face value** (report the value, handling Ace/face-card special cases).
- **Know its suit** (retained as an attribute even if unused in Blackjack).

No need for separate responsibilities for Ace, King, Queen, and Jack — the single `getValue()` method handles all cases.

8.4.4.2 Deck

- **Shuffle** the deck.
- **Deal the next card** (`getNextCard()`).
- **Report how many cards remain** (`getDeckSize()`).
- **Know whether a full deck exists** to begin a game (`resetDeck()` / `completeDeck()`).

8.4.4.3 Hand

- **Know how many cards** are in the hand.
- **Know the total value** of the hand.
- **Show the hand** to the output.

8.4.4.4 Dealer

- **Start a new game** (`startNewGame()`).
- **Get a card** from the deck (`getCard():Card`).
- **Show and evaluate** the dealer's hand.
- **Know the number of cards** in the dealer's hand.
- **Request a card** from the player (hit or hold).
- **Determine the winner** (`-determineWinner():Player`).

8.4.4.5 Player

- **Decide whether to request more cards** (`moreCards():boolean`).
- **Get a card** (`getCard():void`).
- **Show the hand** (`showHand():void`).
- **Get the value of the hand** (`getValueOfHand():int`).

8.4.4.6 Bet (deferred)

The **Bet** class would need to know the type of bet, its monetary value, the player's remaining balance, and whether the bet can be covered. This class is not required for the core game and can be added later without affecting the existing design.

8.4.5 Step 4 — CRC Cards

Definition 8.4.1 (CRC Card). A **CRC** (*Class-Responsibility-Collaborator*) card is a simple index card (or table) with three fields: the class name, its responsibilities, and the other classes it collaborates with. CRC cards were introduced by Kent Beck and Ward Cunningham in their 1989 OOPSLA paper “*A Laboratory for Teaching Object-Oriented Thinking*”.

CRC cards are useful because:

- They are *portable* — no computer is required; design discussions can happen anywhere.
- They allow participants to *role-play* the system: each person holds a card and “acts” as that object, passing messages to other cards. This experiential exercise builds intuition for OO design.
- They are a proven pedagogical tool for teaching the object-oriented paradigm.

The CRC cards for the Blackjack system are summarised in Tables [8.1–8.5](#).

Class: Card	
Responsibility	Collaborators
Get name	Deck
Get value	

Table 8.1: CRC card for **Card**.

Class: Deck	
Responsibility	Collaborators
Reset deck	Dealer
Get deck size	Card
Get next card	
Shuffle deck	
Show deck	

Table 8.2: CRC card for **Deck**.

8.4.6 Step 5 — UML Class Diagram

Translating the CRC cards into a UML Class Diagram yields five classes with the following attributes and methods. Visibility markers follow UML convention: + public, - private, no marker = package/default.

Key relationships between classes:

- Dealer *deals* $\longleftrightarrow$ Deck (association).
- Deck *contains* $52 \times$ Card (composition, multiplicity 52).
- Dealer *shows* $\longrightarrow$ Hand (association, multiplicity 1–1).
- Player *shows* $\longrightarrow$ Hand (association, multiplicity 1–1).
- Dealer *gives card to* $\longrightarrow$ Player (association, multiplicity 1 to 1..*).

Exam Tip: Visibility and Naming

In UML, + denotes **public** and - denotes **private**. Notice that the four private helper methods in **Deck** (`-createHearts()`, etc.) are not exposed in the CRC card — they are implementation details discovered *during* design, not from the requirements.

8.4.7 What's Next: From Design to Code

The CRC cards and UML Class Diagram provide a solid blueprint. A few important caveats:

- The design is **not final**. Changes are expected as implementation exposes further details (missing methods, incorrect multiplicities, etc.).
- Teams should discuss and iterate. Good software design is a collaborative process.
- The design is a *starting point* for writing code, not a guarantee that the code will compile or run correctly on the first attempt.

Class: Dealer	
Responsibility	Collaborators
Start a new game	Hand
Get a card	Player
	Deck

Table 8.3: CRC card for Dealer.

Class: Player	
Responsibility	Collaborators
Want more cards?	Hand
Get a card	Dealer
Show hand	
Get value of hand	

Table 8.4: CRC card for Player.

Warning: Design Completeness

A class diagram and a set of CRC cards tell you *what* objects exist and *what* they can do, but they do not fully specify *how* objects interact in every scenario. Sequence Diagrams must be drawn for each major use case to validate that the class design is sufficient — just as was illustrated in the Professor–Student example in Section 8.3.

Class: Hand	
Responsibility	Collaborators
Return value	Player
Add a card	Dealer
Show hand	

Table 8.5: CRC card for Hand.

Class	Attributes	Methods
Card	-name:String -value:int -suit:String	+getName():String +getValue():int +getSuit():String +setName():void +setValue():void +setSuit():void
Deck	-cards:Vector -shuffledCards:Vector -random:Random -nextItem:int	+resetDeck():void +getDeckSize():int +getNextCard():Card +shuffleDeck():void +showOriginalDeck():void +showShuffledDeck():void +completeDeck():void -createHearts():void, -createSpades():void, -createClubs():void, -createDiamonds():void
Hand	-value:int -hand:Vector	+getValue():int +addToHand():void +showHand():void
Player	limit:int	Player() +moreCards():boolean +getCard():void +showHand():void +getValueOfHand():int
Dealer	—	startNewGame():void +getCard():Card -determineWinner():Player

Table 8.6: Blackjack UML class summary.

Chapter 9

Design Principles

9.1 Introduction

This chapter addresses the question of what separates *good* software design from *bad* software design. A working piece of software may not necessarily be well-designed—it may function today but show symptoms of breaking or design degradation as soon as requirements change. The chapter introduces three symptoms of rotting design, then outlines the core object-oriented design goals (managing coupling, cohesion, and encapsulation) before presenting the **SOLID** design principles that formalise these goals into actionable guidelines.

9.1.1 Learning Objectives

By the end of this chapter, students should be able to:

- Explain the characteristics of good and bad design.
- Describe the various OO design goals including coupling, cohesion, and encapsulation.
- Explain and apply each of the five SOLID design principles.

9.2 Characteristics of Good and Bad Design

Software that “works” today is not necessarily well-designed. Poorly designed software may show *symptoms of design rot* as soon as a change is introduced. Good design, by contrast, produces code that is clean, easy to read, easy to maintain and modify, efficient, reliable, and secure. As Kernighan and Plauger emphasise, good design and programming skill is not learned from generalities alone but from careful study and imitation of well-designed programs.

Exam Tip: “Working” $\neq$ “Well-Designed”

A program may produce correct output but still exhibit rigidity, fragility, or immobility. When evaluating a design, look beyond current functionality to how the system responds to *change*.

9.2.1 Symptoms of Rotting Design

Three classic symptoms indicate that a design is deteriorating:

Definition 9.2.1 (Rigidity). Rigidity is the tendency for software to be difficult to change, even in simple ways. Every change causes a *cascade* of subsequent changes in dependent modules—a ripple or snowball effect that makes even minor modifications unexpectedly costly.

Definition 9.2.2 (Fragility). Fragility is the tendency of software to break in many places every time it is changed. The breakage often occurs in areas that have no conceptual relationship with the area that was modified. This makes the codebase unpredictable and developers reluctant to make changes.

Definition 9.2.3 (Immobility). Immobility is the inability to reuse a software module in other projects or even in other parts of the same project. The module in question has accumulated too many dependencies (“baggage”) that it relies upon, making extraction prohibitively expensive—analogueous to trying to unplug a single cable from a tangled mess of wires.

Warning: Design Rot Compounds

Rigidity, fragility, and immobility are mutually reinforcing. Rigid code is often fragile (cascading changes break unrelated areas) and immobile (heavy coupling prevents extraction). Recognising the *first* symptom early is critical.

9.3 Object-Oriented Design Goals

The overarching objective of OO design is to **minimise the impact of change**. We might want to change a class or package to add new functionality, alter business rules, or improve the design. We might also be *forced* to change a class because a dependency has changed (e.g. a method signature in another class was modified).

Key Result: Core OO Design Objective

Manage dependencies between classes and packages to minimise the impact of change on other parts of the software.

Much of OO design is about managing dependencies. It is very difficult to write OO code without creating a dependency on something; the goal is not to eliminate dependencies entirely but to ensure that each dependency is deliberate and well-managed.

9.3.1 Class Design Guidelines

The central class design guideline is to manage **coupling** and **cohesion** with three considerations:

1. **Reusability** — design with reuse in mind so that components can be employed in multiple contexts.
2. **Extensibility** — design so that new behaviour can be added without modifying existing code.
3. **Maintainability** — design so that defects can be localised and fixed without cascading effects.

The target is *loose (low) coupling* and *high cohesion*.

9.3.2 Designing for Change: Encapsulation, Coupling, and Cohesion

A modular program has **well-defined**, conceptually **simple**, and **independent** units interacting through well-defined **interfaces**. Three mechanisms work together to achieve this:

Definition 9.3.1 (Encapsulation). Encapsulation bundles data and the methods that operate on it within a single unit (class), exposing only a public interface and hiding internal implementation details. This ensures that external code cannot depend on the object’s internal representation.

Definition 9.3.2 (Coupling). Coupling measures the degree of interdependence between classes or modules. *Tight coupling* means that classes know a great deal about each other’s internals; a change in one ripples through many others. *Loose coupling* means that classes interact through narrow, well-defined interfaces, limiting the blast radius of change.

Definition 9.3.3 (Cohesion). Cohesion measures how strongly the responsibilities within a single class are related. A highly cohesive class does one thing and does it well. A class with low cohesion aggregates unrelated responsibilities, making it harder to understand, test, and change.

Exam Tip: Coupling vs. Cohesion

Low coupling is about *between*-class dependencies; high cohesion is about *within*-class focus. Good design aims for loose (low) coupling and high cohesion simultaneously.

9.3.3 Example — Data Persistence

Consider a `DataManager` class responsible for persisting entities.

Bad encapsulation: `DataManager` contains methods like `validateCustomer(cusID, pwd)` that directly compare the password by retrieving it from the database. This means `DataManager` must know the inner workings of `Customer` and `Account`—a violation of encapsulation.

Example 9.3.4 (Bad Encapsulation in `DataManager`).

```
1 class DataManager {
2     boolean validateCustomer(String cusID, String pwd) {
3         // get password from DB matching cusID
4         String passwd = ...;
5         if (passwd.equals(pwd))
6             return true;
7         return false;
8     }
9 }
```

Better encapsulation: Delegate validation to the `Customer` class itself. Now `DataManager` simply asks the `Customer` to validate, and the details of how validation occurs (plain comparison, hashing, etc.) are hidden inside `Customer`.

Example 9.3.5 (Improved Encapsulation via Delegation).

```
1 class DataManager {
2     boolean validateCustomer(String cusID, String pwd) {
3         Customer c = new Customer(cusID);
4         return c.validate(pwd);
5     }
6 }
7
8 class Customer {
9     boolean validate(String pwd) {
10         // compare plain pwd, or hash, or ...
11     }
12 }
```

In the improved design, `Customer` and `Account` are each responsible for their own validation and persistence, while `DataManager` merely coordinates them.

9.3.4 Example — Finance Index Display

Consider a `NASDAQ` class that tracks financial data and a `DataDisplay` class that renders charts.

Bad design: `NASDAQ` holds a `displayType` field and calls specific methods on `DataDisplay` (e.g. `displayDailyData`, `displayMonthlyData`). This suffers from tight coupling (both classes need each other), low cohesion (`NASDAQ` manages both data and display logic), and bad encapsulation (`NASDAQ` must know the internal `displayType` of `DataDisplay`).

Better design: Move `displayType` into `DataDisplay` and let `DataDisplay` pull data from NASDAQ via a `getData()` method. The `DataDisplay` decides how to render the data. Now the dependency direction is reversed: `DataDisplay` depends on NASDAQ's public interface, and NASDAQ no longer knows anything about display concerns.

9.3.5 Example — Bank Account (Interfaces for Loose Coupling)

A `Client` class uses both `Account` and `AccountInfoDisplay`. If these are concrete classes, adding a new account type or display format (e.g. phone display) requires modifying existing code.

By introducing `<<interface>>` types for both `Account` (with methods `getSummary()`, `getTransactionLog()`) and `AccountInfoDisplay` (with method `format()`), the client depends only on abstractions. Concrete classes such as `SavingsAccount`, `CheckingAccount`, `WebDisplay`, `ATMDisplay`, and `PhoneDisplay` implement these interfaces. New types can be added without changing the `Client` at all.

Key Result: Interfaces Promote Loose Coupling

Programming to interfaces rather than concrete classes decouples clients from implementations. New implementations can be added without modifying client code.

9.3.6 Inheritance vs. Delegation (Composition)

Consider a charting library with `ScatterPlot` and `BarChart` that share axis-drawing behaviour.

Inheritance approach: Both extend an abstract `Graph` class (“is-a” relationship). This creates tight coupling—when `Graph` changes, all subclasses must be recompiled.

Delegation (composition) approach: Both hold a reference to an `Axes` object (“has-a” relationship). When `Axes` changes its internals, the chart classes do *not* need to be recompiled, because they depend only on `Axes`'s public interface.

Example 9.3.6 (Inheritance vs. Delegation in Java).

```
1 // Inheritance: BarChart recompiled when Graph changes
2 class BarChart extends Graph { ... }
3
4 // Delegation: BarChart NOT recompiled when Axes changes
5 class BarChart {
6     private Axes myAxes;
7     ...
8 }
```

Exam Tip: Favour Composition Over Inheritance

The Gang of Four principle “favour object composition over class inheritance” reduces coupling. Use inheritance for genuine “is-a” relationships (e.g. `Lecturer` is-a `Person`) and delegation for shared functionality that does not represent a subtype relationship (e.g. a chart “has” axes).

9.3.7 Inheritance vs. Role

Not every categorisation warrants a subclass. Roles such as “Father” and “Mother” are not proper subtypes of `Person`—a person can hold multiple family roles simultaneously, and roles can change over time. The correct design is to model role as an attribute (e.g. an `enum FAMILY_ROLE`)

or via a separate `Role` interface, rather than creating `Father extends Person`, `Mother extends Person`, etc.

9.4 SOLID Design Principles

The SOLID principles are five class-level design guidelines introduced by Robert C. Martin that together promote loose coupling, high cohesion, and resilience to change. Each letter stands for one principle:

Letter	Principle	Core Idea
S	Single Responsibility (SRP)	One reason to change
O	Open–Closed (OCP)	Open for extension, closed for modification
L	Liskov Substitution (LSP)	Subtypes substitutable for base types
I	Interface Segregation (ISP)	Many specific interfaces > one fat interface
D	Dependency Inversion (DIP)	Depend on abstractions, not details

9.4.1 Single Responsibility Principle (SRP)

Definition 9.4.1 (Single Responsibility Principle). There should never be more than **one reason** for a class to change. Each responsibility is an axis of change; if a class assumes more than one responsibility, those responsibilities become coupled.

SRP is directly related to cohesion (Definition 9.3.3). A class that handles both data storage *and* display logic, for example, has two reasons to change: a change in persistence technology or a change in UI requirements. Such a class should be split into two, each with a single responsibility.

Warning: Avoid “God” Classes

A “God” class is one that accumulates so many responsibilities that it becomes the centre of the system. Every change touches it, and it depends on everything else. SRP is the primary defence against God classes.

9.4.2 Open–Closed Principle (OCP)

Definition 9.4.2 (Open–Closed Principle). A module should be **open for extension** but **closed for modification**. We want to change what the module does without changing its source code. Abstraction is the key to OCP.

This principle, originated from the work of Bertrand Meyer, encourages designs where new behaviour is added by writing *new* classes (extensions) rather than editing existing ones.

Example 9.4.3 (OCP — Transport Fare Calculation). A `Transport` class with a `calcFare()` method that uses a `switch(type)` over `BUS`, `TAXI`, `MRT` violates OCP: adding a new transport type (e.g. a tricycle) requires modifying the existing `switch` statement.

The OCP-compliant design makes `Transport` an abstract class with an abstract `calcFare()` method. Each transport type—`Bus`, `Taxi`, `MRT`—is a subclass that overrides `calcFare()`. Adding a tricycle simply means adding a new subclass; the existing `Transport` class and all existing subclasses remain unchanged.

Key Result: Abstraction Enables OCP

Use abstract classes or interfaces to define stable contracts. New behaviours are introduced through new subclasses or implementors, not by editing existing code.

9.4.3 Liskov Substitution Principle (LSP)

Definition 9.4.4 (Liskov Substitution Principle). If S is a subtype of T , then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program. Informally: *subtypes must be substitutable for their base types*.

Barbara Liskov first formulated this principle in 1988. A user of a base class should continue to function properly if a derivative of that base class is passed to it.

9.4.3.1 Design by Contract

LSP can be restated in terms of *Design by Contract*: a method's public interface should specify **pre-conditions** (what must be true before execution) and **post-conditions** (what must be true after execution). A derived class is substitutable for its base class if and only if:

- a) Its pre-conditions are **no stronger** than the base class method (*expect no more*).
- b) Its post-conditions are **no weaker** than the base class method (*provide no less*).

Example 9.4.5 (LSP Violation — AccountInfoDisplay). The base class `AccountInfoDisplay` checks for `null` internally and returns full account details. A subclass `ATMDisplay` violates LSP in two ways:

1. It *expects more*—it assumes the client has already checked for `null` (stronger pre-condition).
2. It *provides less*—it returns only partial account details (weaker post-condition).

Substituting an `ATMDisplay` where `AccountInfoDisplay` is expected could cause a `NullPointerException` or display incomplete information.

Example 9.4.6 (LSP Violation — Square extends Rectangle). A classic LSP violation: `Square` extends `Rectangle` and overrides `setWidth` and `setHeight` to keep both dimensions equal. Client code that calls `setWidth(5)` then `setHeight(10)` on what it believes is a `Rectangle` will compute `getArea()` as 100 instead of the expected 50, because the `Square` silently coupled width and height.

The key lesson: mathematical subtype relationships (a square *is* a rectangle) do not always translate into correct inheritance hierarchies when mutability is involved.

Warning: LSP and Mutability

If a subclass changes the semantics of an inherited setter (e.g. coupling two independent properties), it violates LSP even though the “is-a” relationship seems correct in the real world. Consider using composition or making the types immutable instead.

9.4.4 Interface Segregation Principle (ISP)

Definition 9.4.7 (Interface Segregation Principle). Many client-specific interfaces are better than one general-purpose interface. Classes should not depend on interfaces that they do not use. Avoid *fat* interfaces.

Example 9.4.8 (ISP — Animal Interfaces). A single `IAntimal` interface with methods `fly()`, `run()`, and `swim()` forces a `Tiger` to provide empty implementations of `fly()` and `swim()`, and a `Dolphin` to stub out `fly()` and `run()`.

Applying ISP, we split into three narrow interfaces: `IFlyable` (with `fly()`), `IRunnable` (with `run()`), and `ISwimmable` (with `swim()`). Each animal implements only the interfaces relevant to it:

- Goose implements IFlyable, IRunnable, and ISwimmable.
- Tiger implements IRunnable only.
- Dolphin implements ISwimmable only.
- Crocodile implements IRunnable and ISwimmable.

No class is forced to depend on methods it does not use.

9.4.5 Dependency Inversion Principle (DIP)

Definition 9.4.9 (Dependency Inversion Principle).

- High-level modules should not depend upon low-level modules. Both should depend upon abstractions.
- Abstractions should not depend upon details. Details should depend upon abstractions.

When high-level modules are independent of low-level modules, the high-level modules can be reused straightforwardly. DIP is the principle at the heart of *framework* design and is also known as **Inversion of Control (IoC)**.

Example 9.4.10 (DIP — Disc Player Framework). A `DiscPlayerApp` depends on abstract `Disc` and `DiscPlayer` classes rather than concrete `DVDDisc` or `BlurayDisc`. An `ObjectCreator` uses reflection (or a configuration file) to instantiate the correct concrete disc type at runtime. Adding a new disc format (e.g. VCD) requires only creating a new subclass and updating the configuration—existing high-level code is unchanged.

Key Result: Depend on Abstractions

Both high-level policy modules and low-level detail modules should depend on abstractions (interfaces or abstract classes). This inverts the traditional top-down dependency chain and enables frameworks, plugin architectures, and easy testing via mocks.

9.4.5.1 Frameworks and Inversion of Control

A **framework** is a large body of prewritten classes to which you add your own classes by extending or implementing the framework's classes and interfaces. The framework is in control: it instantiates your class and calls your methods (callbacks). You plug into the framework via the interfaces and abstract classes it defines, which is a direct application of DIP.

9.5 Design Patterns (Overview)

Design patterns are proven solutions to commonly recurring design problems. The *Gang of Four* (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides) catalogued 23 foundational patterns in their seminal 1994 book. Examples include Singleton, Adapter, Observer, and many more. A widely-used architectural pattern is **Model–View–Controller (MVC)**, especially prevalent in web application development.

Design patterns build on the SOLID principles—for instance, the Observer pattern embodies OCP and DIP, while the Strategy pattern is a direct application of OCP via composition.

Exam Tip: Connecting Principles to Patterns

When asked about a design pattern, identify which SOLID principles it supports. For example, the Strategy pattern lets you extend behaviour without modifying the context class (OCP), and the context depends on an abstract strategy interface (DIP).

9.6 Summary

Good OO design aims to minimise the impact of change by managing dependencies through encapsulation, low coupling, and high cohesion. The five SOLID principles provide concrete, actionable guidelines:

1. **SRP**: One class, one reason to change.
2. **OCP**: Extend behaviour via new subclasses; do not modify existing code.
3. **LSP**: Subtypes must be drop-in replacements for their base types (expect no more, provide no less).
4. **ISP**: Prefer many small, focused interfaces over one large interface.
5. **DIP**: Depend on abstractions, not concrete implementations.

These principles, combined with the preference for composition over inheritance and the use of design patterns, form the foundation of maintainable, extensible, and reusable object-oriented software.

Chapter 10

Modern Java Enhancements

Java has evolved significantly since its initial release, with major enhancements arriving in Java 5, Java 8, and Java 12+. This chapter covers the key modern features introduced across those versions: Generics, Collections enhancements, the enhanced `for` loop, Lambda Expressions, the Streams API, default methods in interfaces, and the enhanced `switch` statement.

Java's progression can be summarised as: Java 1.x established the foundations; Java 5 brought major language improvements (Generics, autoboxing, enhanced `for`, annotations); Java 8 introduced functional programming via Lambdas, Streams, and default methods; Java 12+ refined syntax with the enhanced `switch` expression.

10.1 Introduction to Generics (Java 5)

10.1.1 Motivation: The Problem Without Generics

Before Java 5, collections used *raw types* — every element was stored as `Object`. Retrieving an element required an explicit cast, which compiled without error even when the cast was wrong, producing a `ClassCastException` at runtime.

Example 10.1.1 (Raw-type list (pre-Java 5)).

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 List myList = new ArrayList(); // raw type -- no type safety
5 myList.add("Java");
6 myList.add(2); // Integer added by mistake
7
8 String str = (String) myList.get(0); // OK
9 // Compiles but throws ClassCastException at runtime:
10 String num = (String) myList.get(1);
```

Warning: Raw Types

Using raw types silences the compiler. A wrong cast compiles silently but explodes at runtime with `ClassCastException`. Always use parameterised (generic) types.

10.1.2 Generics: Type-Safe Collections

Java 5 introduced *Generics*, allowing a type parameter to be specified at the declaration site. The compiler enforces type correctness, and no explicit cast is needed when retrieving elements.

Example 10.1.2 (Generic list (Java 5+)).

```
1 List<String> myList = new ArrayList<>(); // type-safe
2 myList.add("Java");
3 // myList.add(2); -- compile-time error: wrong type
```

```

4
5 String str = myList.get(0); // no cast needed

```

The type parameter `E` in `ArrayList<E>` is replaced by a concrete type (e.g. `String`) at the use site. The generic definition of `ArrayList` looks like:

```

1 public class ArrayList<E> implements Iterable<E> {
2     private Object[] elements;
3     private int size = 0;
4
5     public boolean add(E element) {
6         ensureCapacity(size + 1);
7         elements[size++] = element;
8         return true;
9     }
10    // get, remove, set, ...
11 }

```

10.1.2.1 The Diamond Operator (Java 7)

Java 7 introduced the *diamond operator* `<>`, enabling type inference on the right-hand side and eliminating redundancy:

```

1 // Java 5 -- explicit type on both sides:
2 List<String> list = new ArrayList<String>();
3
4 // Java 7+ -- type inferred:
5 List<String> list = new ArrayList<>();

```

10.1.3 Custom Generic Classes

A generic class is declared with a type parameter in angle brackets:

Example 10.1.3 (`Box<T>` – single type parameter).

```

1 public class Box<T> {
2     private T content;
3     public void set(T content) { this.content = content; }
4     public T get() { return content; }
5 }
6
7 Box<Integer> intBox = new Box<>();
8 intBox.set(123);
9 Integer number = intBox.get(); // returns Integer, no cast

```

Multiple type parameters are also supported:

Example 10.1.4 (`Box<T,U>` – two type parameters).

```

1 class Box<T, U> {
2     private T first;
3     private U second;
4     public Box(T first, U second) {
5         this.first = first;
6         this.second = second;
7     }
8     public T getFirst() { return first; }
9     public U getSecond() { return second; }
10 }

```

```

11
12 Box<String, Double> priceBox = new Box<>("Laptop", 799.99);
13 // Item: Laptop, Price: $799.99
14 System.out.println("Item: " + priceBox.getFirst()
15     + ", Price: $" + priceBox.getSecond());

```

10.1.4 Wildcards

Wildcards allow methods to accept collections with unknown or bounded type parameters.

10.1.4.1 Unbounded Wildcard (<?>)

Use <?> when the element type is not relevant — the method only reads elements as `Object`.

Example 10.1.5 (Printing any list).

```

1 public static void printList(List<?> list) {
2     for (Object element : list) {
3         System.out.println(element);
4     }
5 }
6 // Works for List<String>, List<Integer>, etc.

```

10.1.4.2 Bounded Wildcards

Definition 10.1.6 (Upper Bounded Wildcard). `List<? extends Fruit>` accepts `List<Fruit>`, `List<Apple>`, `List<Banana>`, or any subtype of `Fruit`. Used when *reading* from the collection.

Definition 10.1.7 (Lower Bounded Wildcard). `List<? super Fruit>` accepts `List<Fruit>`, `List<Food>`, or any supertype of `Fruit`. Used when *writing* to the collection.

10.1.5 Type Parameter Naming Conventions

The Java standard follows single-letter, upper-case type parameter names: `E` (Element), `K` (Key), `V` (Value), `N` (Number), `T` (Type), and `S`, `U`, `V` for additional parameters.

Key Result: Benefits of Generics

Generics provide (1) **compile-time type safety** — wrong types are caught before runtime; (2) **no explicit casting** on retrieval; (3) **reusable code** — one generic class serves many types.

10.2 Collections Enhancements (Java 5+)

Java's Collections Framework provides a unified architecture for storing and manipulating groups of objects. The top-level hierarchy is:

The root interface `Iterable<E>` enables the enhanced `for` loop. `Collection<E>` extends it and is the parent of the three main sub-interfaces: **List** (ordered, duplicates allowed — `ArrayList`, `LinkedList`, `Vector/Stack`), **Queue** (FIFO ordering — `PriorityQueue`, `Deque/ArrayDeque`), and **Set** (no duplicates — `HashSet`, `LinkedHashSet`, `SortedSet/TreeSet`). The **Map** interface (`HashMap`, `LinkedHashMap`, `Hashtable`, `TreeMap`) is separate from `Collection` but still part of the framework.

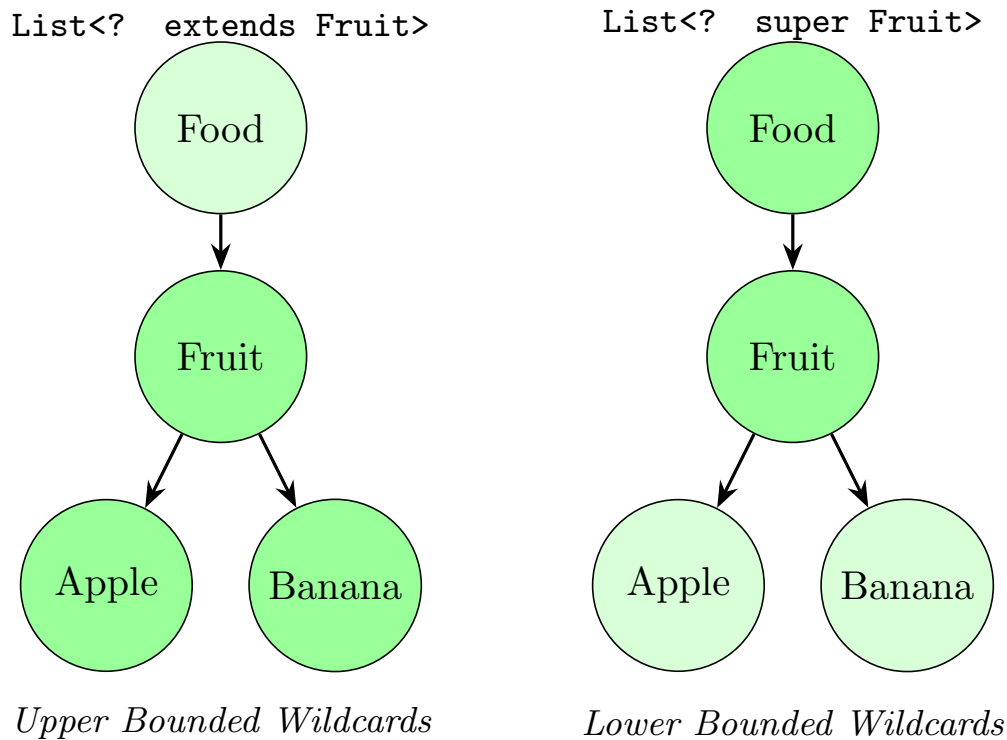


Figure 10.1: Java generic wildcards. Shaded nodes indicate which types are accepted by each wildcard form.

With Generics, all collection types became parameterised in Java 5, replacing `add(Object)` with `add(E)` and `get(int):Object` with `get(int):E`. This eliminated unsafe casts throughout collection usage.

10.3 Enhanced For Loop (Java 5)

Java 5 introduced four main ways to traverse a collection, each suited to different needs.

10.3.1 Basic For Loop (Index-Based)

```
1 List<String> list = Arrays.asList("Apple", "Banana", "Cherry");
2 for (int i = 0; i < list.size(); i++) {
3     System.out.println(list.get(i));
4 }
```

Best for index-based access (e.g. when the position of the element matters).

10.3.2 Enhanced For-Each Loop

Syntax: `for (Type variable : collection) { ... }`

```
1 List<String> list = Arrays.asList("Apple", "Banana", "Cherry");
2 for (String fruit : list) {
3     System.out.println(fruit);
4 }
```

The loop variable (`fruit`) holds the current element; `Type` is its declared type; `collection` is any array or `Iterable`. Modifications to the collection are not permitted during this loop.

10.3.3 Iterator (Safe Element Removal)

```
1 Iterator<String> iterator = list.iterator();
2 while (iterator.hasNext()) {
3     String fruit = iterator.next();
4     System.out.println(fruit);
5     if (fruit.equals("Banana")) {
6         iterator.remove(); // safe removal during iteration
7     }
8 }
```

Warning: ConcurrentModificationException

Never call `list.remove()` inside an enhanced for loop or while iterating with an `Iterator`. Always use `iterator.remove()` to delete elements during iteration.

10.3.4 Stream API `forEach` (Functional, Java 8)

```
1 list.stream().forEach(item -> System.out.println("Element: " + item));
```

`.stream()` creates a stream; `.forEach()` consumes each element via a lambda. Covered in detail in Section 10.5.

10.4 Lambda Expressions (Java 8)

10.4.1 Motivation

Before Java 8, passing behaviour required creating a full class or an anonymous inner class. Lambda expressions reduce this to a concise one-liner.

Example 10.4.1 (Three equivalent approaches). Traditional named class:

```
1 public class HelloWorld {
2     public void sayHello() { System.out.println("Hello, World!"); }
3 }
```

Anonymous inner class:

```
1 Greeting greeting = new Greeting() {
2     public void sayHello() { System.out.println("Hello, World!"); }
3 };
4 greeting.sayHello();
```

Lambda expression:

```
1 Greeting greeting = () -> System.out.println("Hello, World!");
2 greeting.sayHello();
```

10.4.2 Syntax

Definition 10.4.2 (Lambda Expression). A lambda expression has the form:

$$(\text{arguments}) \rightarrow \{ \text{body} \}$$

consisting of an *argument list*, the **arrow token** `->`, and a *body*.

Common forms:

```
1 // No arguments
2 () -> System.out.println("Hello")
3
4 // One argument (parentheses optional)
5 s -> System.out.println(s)
6
7 // Two arguments
8 (x, y) -> x + y
9
10 // Explicit argument types
11 (Integer x, Integer y) -> x + y
12
13 // Multiple statements -- braces and return required
14 (x, y) -> {
15     System.out.println(x);
16     System.out.println(y);
17     return x + y;
18 }
```

10.4.3 Functional Interfaces

A lambda can only be assigned to a *functional interface* — an interface with exactly one abstract method (annotated `@FunctionalInterface`). Examples from `java.util.function`: `Runnable` (`run()`), `Consumer<T>` (`accept(T)`), `BiConsumer<T,U>` (`accept(T,U)`), `Function<T,R>` (`apply(T)`), `BiFunction<T,U,R>`.

Example 10.4.3 (`BiConsumer` with a lambda).

```
1 import java.util.function.BiConsumer;
2
3 BiConsumer<Integer, String> myLambda =
4     (arg1, arg2) -> System.out.println(
5         "Two arguments: " + arg1 + " and " + arg2);
6
7 myLambda.accept(5, "Hello");
8 // Output: Two arguments: 5 and Hello
```

10.4.4 Method References

When a lambda simply delegates to an existing method, a *method reference* (using `::`) is more concise. There are five kinds:

Type	Example	Lambda equivalent
Static	<code>Integer::parseInt</code>	<code>str -> Integer.parseInt(str)</code>
Bound instance	<code>Instant.now()::isAfter</code>	<code>t -> Instant.now().isAfter(t)</code>
Unbound instance	<code>String::toLowerCase</code>	<code>str -> str.toLowerCase()</code>
Class constructor	<code>TreeMap<K,V>::new</code>	<code>() -> new TreeMap<K,V>()</code>
Array constructor	<code>int[]::new</code>	<code>len -> new int[len]</code>

Exam Tip: Lambda Best Practices

Keep lambdas short and focused on a single action. Use method references wherever possible (`System.out::println` instead of `x -> System.out.println(x)`). Avoid placing complex logic inside a lambda body — extract it into a named method.

10.5 Streams API (Java 8)

10.5.1 What is a Stream?

Definition 10.5.1 (Java Stream). A **stream** is a sequence of elements from a source (array, collection, I/O channel) that supports aggregate, functional-style operations via a *stream pipeline*: source $\rightarrow$ intermediate operations $\rightarrow$ terminal operation.

The pipeline model is: a *source* generates a stream instance; zero or more *intermediate operations* (lazy, return a new stream) transform the data; a single *terminal operation* (eager, triggers execution) produces a result.

Key properties of streams are shown in Table 10.1.

Feature	Description
No stored data	Process data from existing sources; never store it.
Immutable processing	Original data is not modified; a new result is produced.
Lazy evaluation	Intermediate operations execute only when a terminal operation is invoked.
Method chaining	Operations return streams, enabling concise pipeline syntax.
Intermediate ops	<code>filter()</code> , <code>map()</code> , <code>sorted()</code> , <code>distinct()</code> — return a stream.
Terminal ops	<code>collect()</code> , <code>forEach()</code> , <code>reduce()</code> , <code>count()</code> — produce a result.
Parallel processing	<code>.parallelStream()</code> splits work across threads automatically.

Table 10.1: Key features of Java Streams.

10.5.2 Stream Operations

Example 10.5.2 (`filter` — keep elements matching a predicate).

```
1 List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
2 List<String> filtered = names.stream()
3   .filter(name -> name.startsWith("A"))
4   .collect(Collectors.toList()); // [Alice]
```

Example 10.5.3 (`map` — transform each element).

```
1 List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
2 List<Integer> squared = numbers.stream()
3   .map(n -> n * n)
4   .collect(Collectors.toList()); // [1, 4, 9, 16]
```

Example 10.5.4 (`sorted` — sort in natural order).

```
1 List<String> sortedNames = names.stream()
2   .sorted()
3   .collect(Collectors.toList());
```

Example 10.5.5 (`forEach` — terminal operation, consumes each element).

```
1 names.stream().forEach(System.out::println);
```

Example 10.5.6 (`collect` — aggregate into a collection).

```

1 List<String> result = names.stream()
2   .filter(name -> name.length() > 3)
3   .collect(Collectors.toList());

```

Example 10.5.7 (Parallel stream).

```

1 List<Integer> parallelResult = numbers.parallelStream()
2   .map(n -> n * n)
3   .collect(Collectors.toList());

```

Example 10.5.8 (Chained pipeline — filter, map, sort, distinct).

```

1 List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");
2 List<String> finalResult = names.stream()
3   .filter(n -> n.startsWith("A"))
4   .map(String::toUpperCase) // method reference
5   .sorted()
6   .distinct()
7   .collect(Collectors.toList()); // [ALICE]

```

Key Result: Streams vs. Loops

Streams do not replace loops entirely, but excel at data transformation pipelines where multiple filter/map/sort operations are chained. Lazy evaluation means unused intermediate results are never computed. Use `parallelStream()` for large datasets, but be aware of thread-safety constraints on shared state.

10.6 Default Methods in Interfaces (Java 8)

10.6.1 What is a Default Method?

Definition 10.6.1 (Default Method). A **default method** is a method declared inside an interface with the `default` keyword and a concrete body. Implementing classes inherit the default implementation and may override it.

Example 10.6.2 (Vehicle interface with a default method).

```

1 interface Vehicle {
2   void start(); // abstract -- must be implemented
3
4   default void stop() {
5     System.out.println("Vehicle is stopping.");
6   }
7 }

```

10.6.2 Why Are Default Methods Needed?

Before Java 8, adding a new method to a published interface broke all existing implementations — they would no longer compile. Default methods allow interfaces to evolve (e.g. the `forEach` and `stream` methods added to `Collection` in Java 8) without breaking code that already implements them.

Example 10.6.3 (Extending `Printer` without breaking implementations).

```

1 interface Printer {
2     void print();
3
4     // New in v2 -- existing implementors are unaffected:
5     default void scan() {
6         System.out.println("Scanning document...");
7     }
8 }

```

10.6.3 Resolving Default Method Conflicts (Multiple Inheritance)

A class may implement two interfaces that both declare a default method with the same signature. The class *must* override that method to resolve the ambiguity, and may call either interface's version via `InterfaceName.super.method()`:

Example 10.6.4 (Resolving a default method conflict).

```

1 interface A {
2     default void show() { System.out.println("A"); }
3 }
4 interface B {
5     default void show() { System.out.println("B"); }
6 }
7 class C implements A, B {
8     @Override
9     public void show() {
10        System.out.println("Resolving conflict");
11        A.super.show(); // explicitly delegates to A
12    }
13 }

```

10.6.4 Static Methods in Interfaces

Java 8 also allows `static` methods in interfaces. They are not inherited by implementing classes and must be called via the interface name:

```

1 interface Utility {
2     static void displayMessage() {
3         System.out.println("Static interface method.");
4     }
5 }
6 // Call: Utility.displayMessage();

```

Key Result: Interface Method Types (Java 8+)

Feature	Purpose	Overridable?
Abstract method	Must be implemented by the class	Yes
Default method	Adds new methods without breaking existing code	Yes
Static method	Utility/helper function on the interface	No

10.7 Enhanced Switch (Java 12+)

10.7.1 Limitations of the Traditional Switch

The traditional `switch` statement has two well-known pitfalls: *fall-through* (execution continues past a matched case unless `break` is written) and verbosity (a separate `break` and assignment are required for each case).

```
1 int num;
2 switch (day) {
3     case MONDAY: num = 1; break;
4     case TUESDAY: num = 2; break;
5     default: num = 0;
6 }
```

10.7.2 Enhanced Switch Expression

Java 12 introduced *switch expressions* with **arrow labels** (`->`), which eliminate fall-through and can be used directly as expressions:

```
1 int num = switch (day) {
2     case MONDAY -> 1;
3     case TUESDAY -> 2;
4     default -> 0;
5 };
```

Multiple labels can be combined in one case:

```
1 int result = switch (grade) {
2     case 'A', 'B' -> 100;
3     case 'C' -> 75;
4     default -> 50;
5 };
```

10.7.3 The yield Statement

When a case branch requires multiple statements, use a block with `yield` to return a value:

```
1 int finalScore = switch (grade) {
2     case 'A' -> {
3         System.out.println("Excellent");
4         yield 100; // returns value from block
5     }
6     case 'B' -> 85;
7     default -> 50;
8 };
```

10.7.4 Using Enhanced Switch with Enums

The enhanced switch pairs naturally with enums, as all cases are known at compile time:

```
1 enum Day { MONDAY, TUESDAY, WEDNESDAY,
2             THURSDAY, FRIDAY, SATURDAY, SUNDAY }
3
4 String activity = switch (day) {
5     case SATURDAY, SUNDAY -> "Relax";
6     default -> "Work";
7 };
```

Key Result: Enhanced Switch Benefits

- **No fall-through** — each arrow case is independent.
- **Expression support** — the result can be assigned directly to a variable.
- **Concise syntax** — fewer lines, no repeated **break**.
- **Exhaustiveness** (with enums/sealed types) — the compiler warns if cases are missing.

10.8 Summary

Java version	Feature	Main benefit
Java 5	Generics	Compile-time type safety; no explicit casts
Java 5	Collections enhancements	Parameterised collections (<code>List<E></code> , etc.)
Java 5	Enhanced for	Cleaner iteration over arrays and collections
Java 7	Diamond operator	Type inference on the right-hand side
Java 8	Lambda expressions	Concise inline behaviour; functional interfaces
Java 8	Streams API	Declarative data pipelines with lazy evaluation
Java 8	Default methods	Interface evolution without breaking existing code
Java 12+	Enhanced switch	Expression-style switch; no fall-through